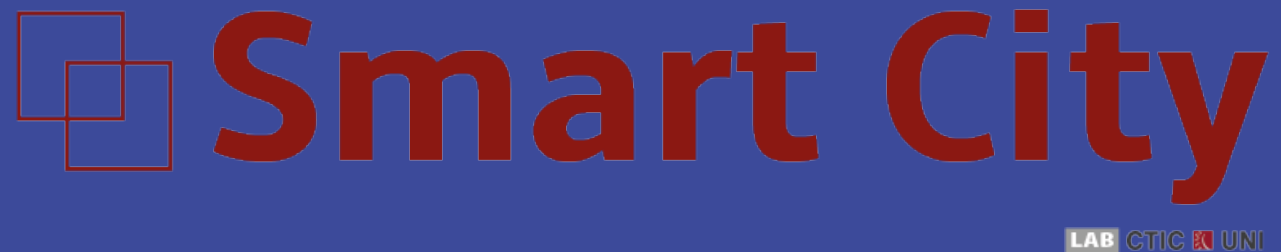
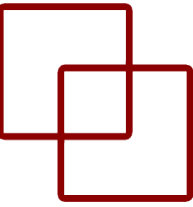


Redes Neurnoales Recurrentes



Dr. Manuel Castillo-Cara
Deep Learning con Keras y Python
Web: www.smartcityperu.org

Índice



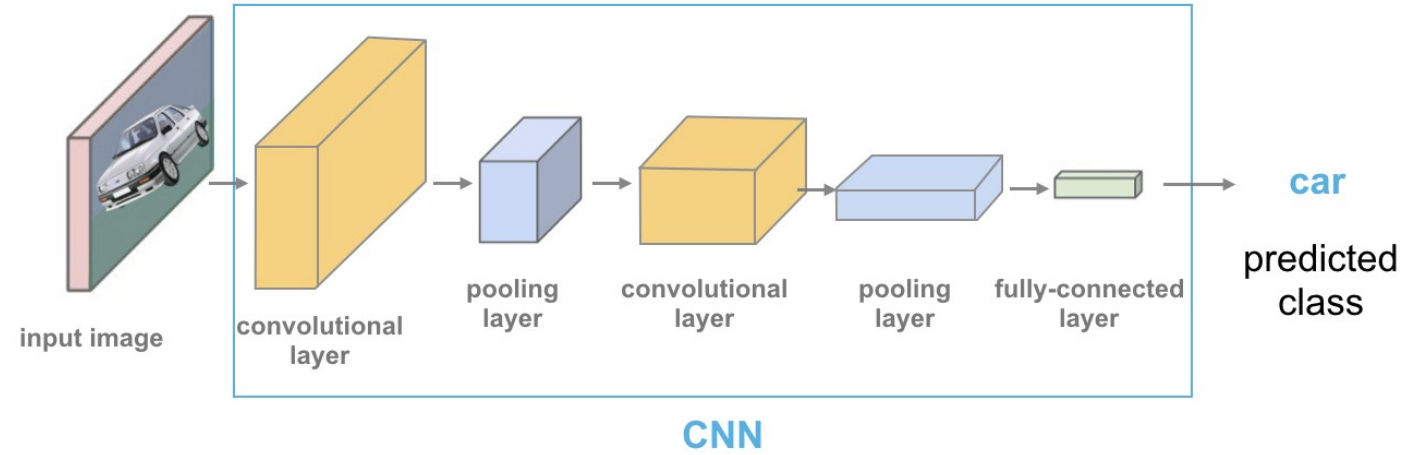
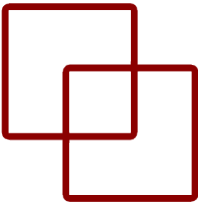
- Background
- Introducción
- Redes neuronales recurrentes (RNNs)
 - Historia
 - Funcionamiento
 - LSTM
- Deep Reinforcement Learning (DRL)
 - Aprendizaje por refuerzo
 - Q-networks y algoritmos Actor-Crítico
 - Aplicaciones
- Conclusiones



Background

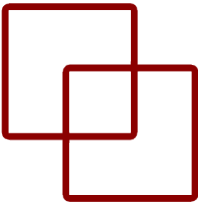
1. Background

CNN vs. RNN

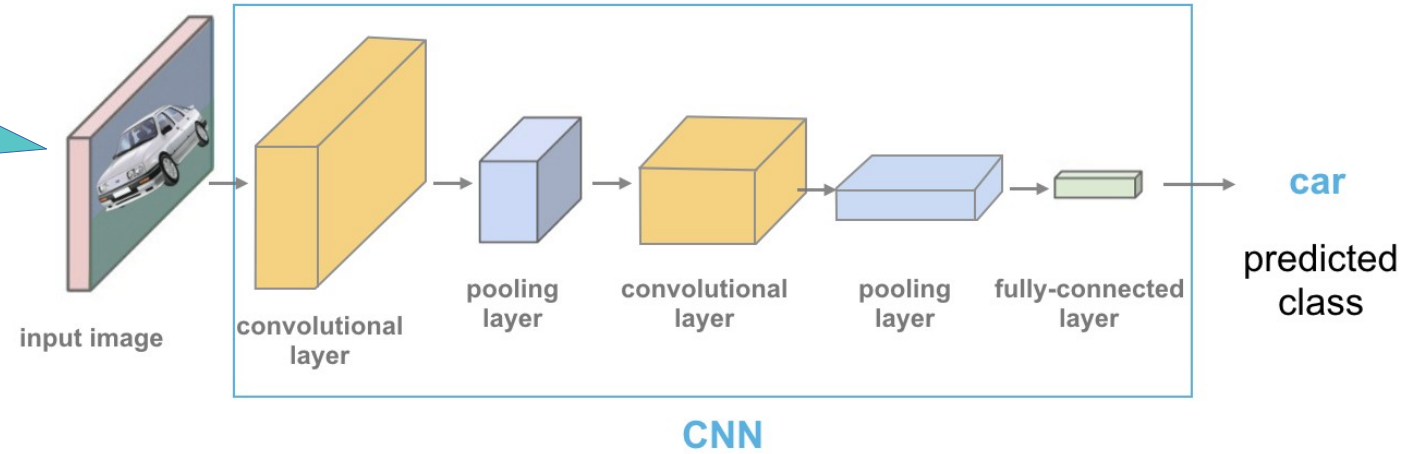


1. Background

CNN vs. RNN

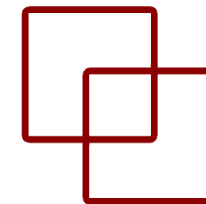


Dato (imagen, secuecnia, palabra, etc) a clasificar
¿Qué pasa si se introduce una secuencia?

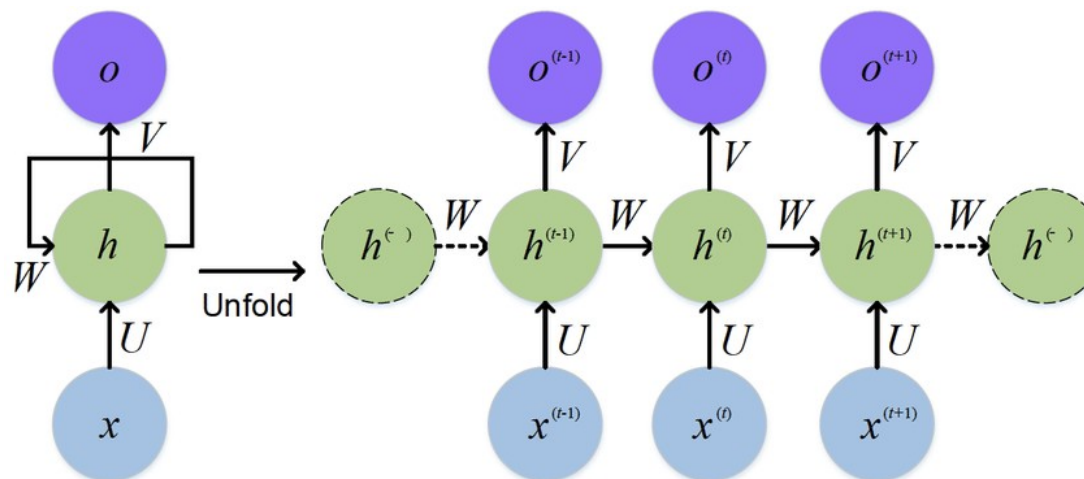
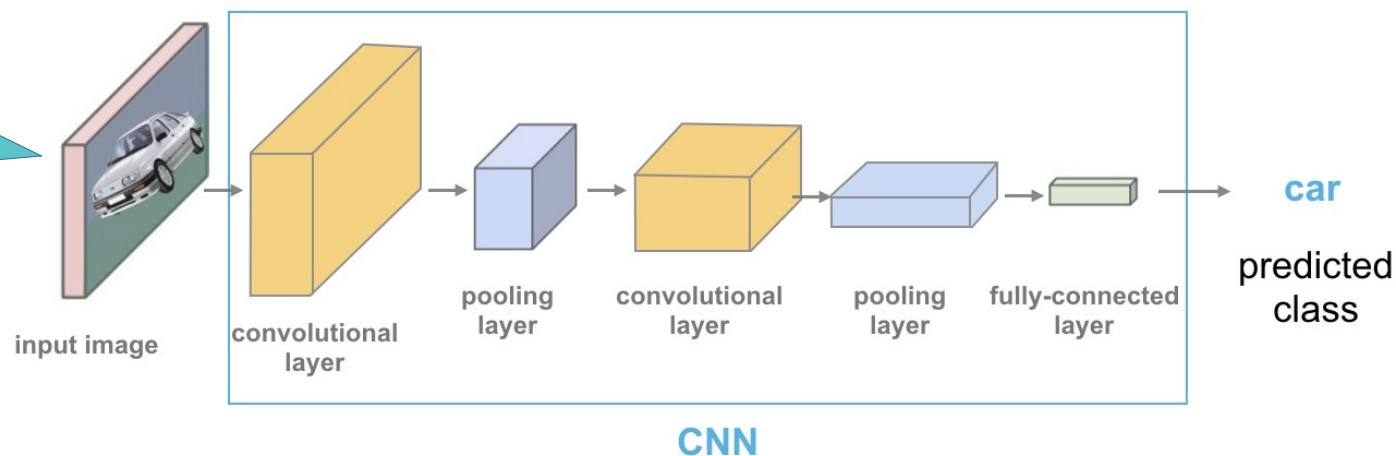


1. Background

CNN vs. RNN

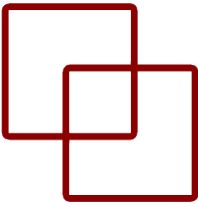


Dato (imagen, secuecnia, palabra, etc) a clasificar
¿Qué pasa si se introduce una secuencia?

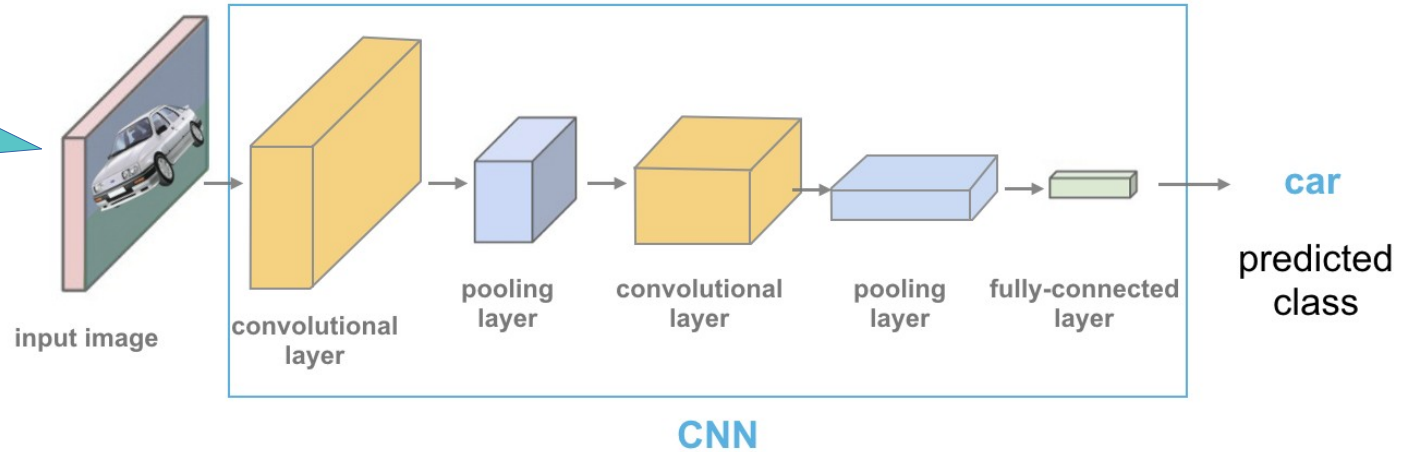


1. Background

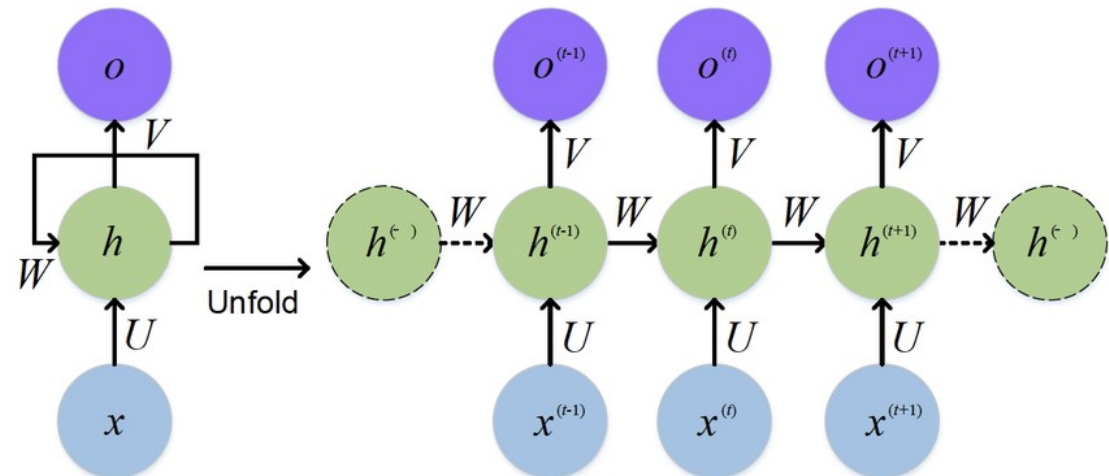
CNN vs. RNN



Dato (imagen, secuencia, palabra, etc) a clasificar
¿Qué pasa si se introduce una secuencia?

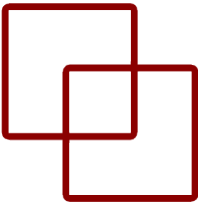


Secuencia (conversación, texto, vídeo) de datos con orden
Los datos están correlacionados dependiendo del texto anterior

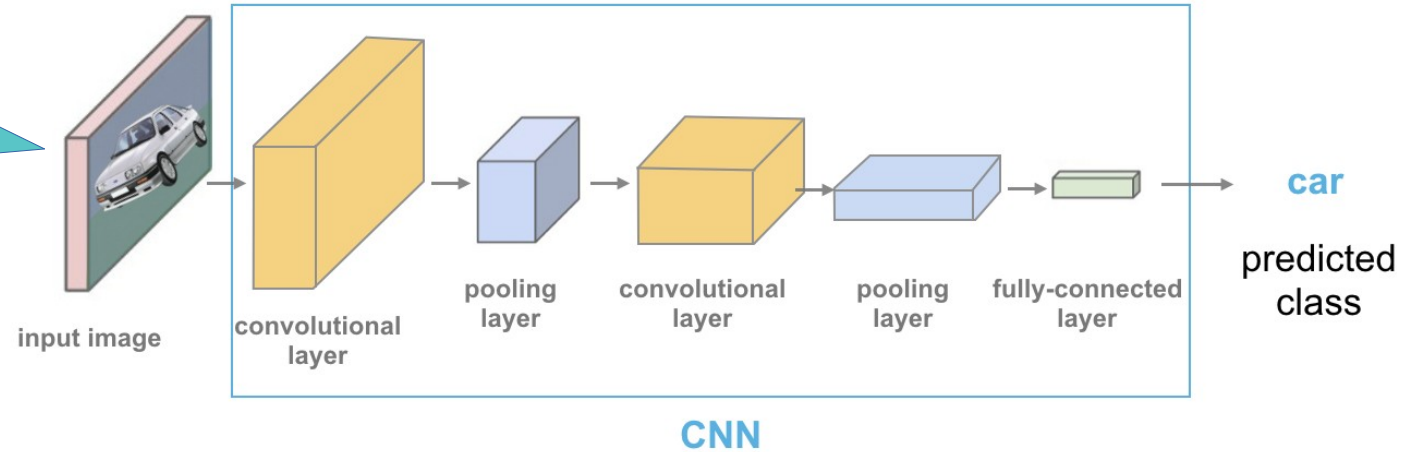


1. Background

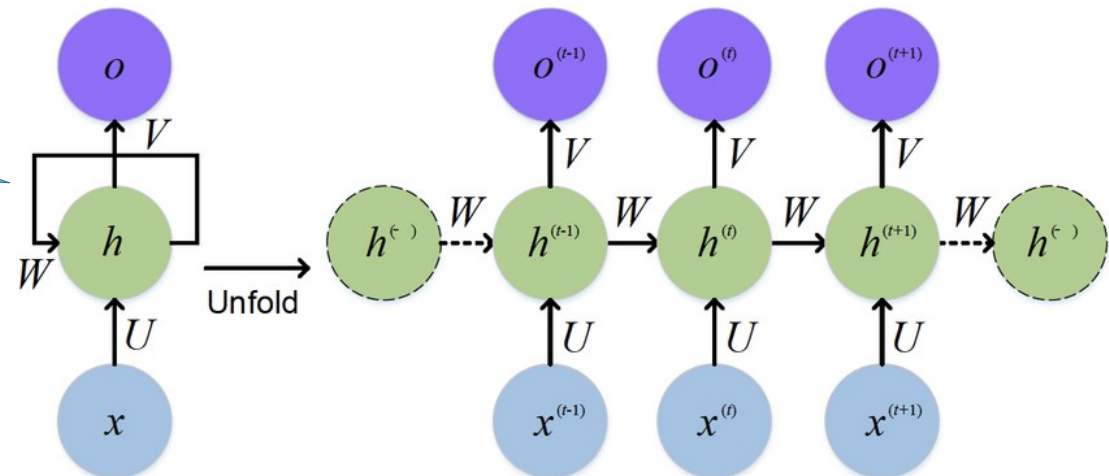
CNN vs. RNN



Dato (imagen, secuencia, palabra, etc) a clasificar
¿Qué pasa si se introduce una secuencia?

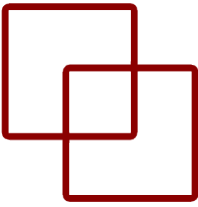


Para ello utiliza la activación de la **iteración anterior** para dar nuevos resultados, i.e., tiene memoria

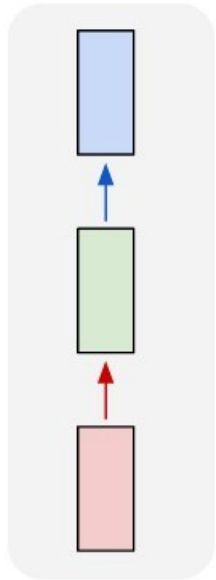


1. Background

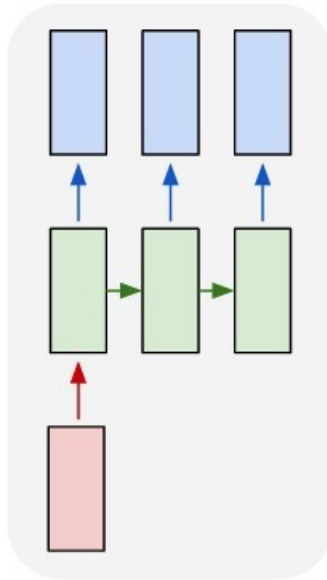
Soporte para secuencias en NN



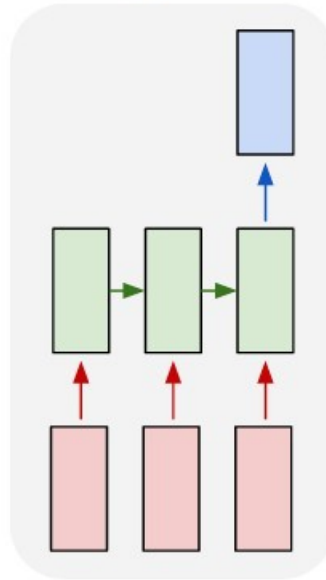
one to one



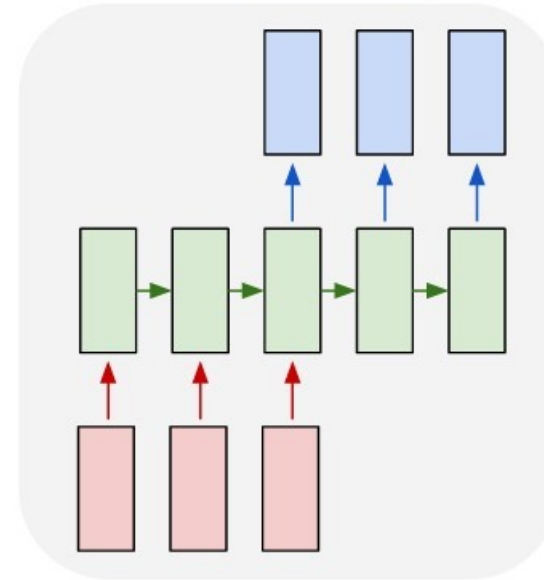
one to many



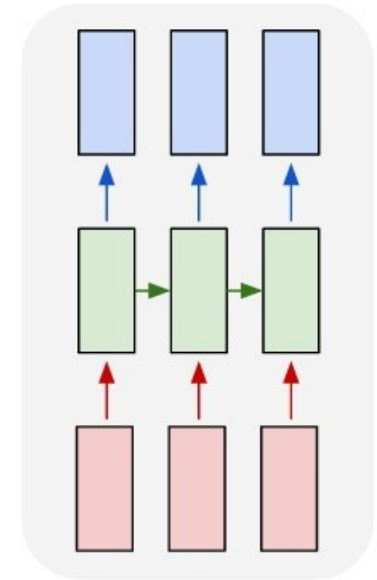
many to one



many to many



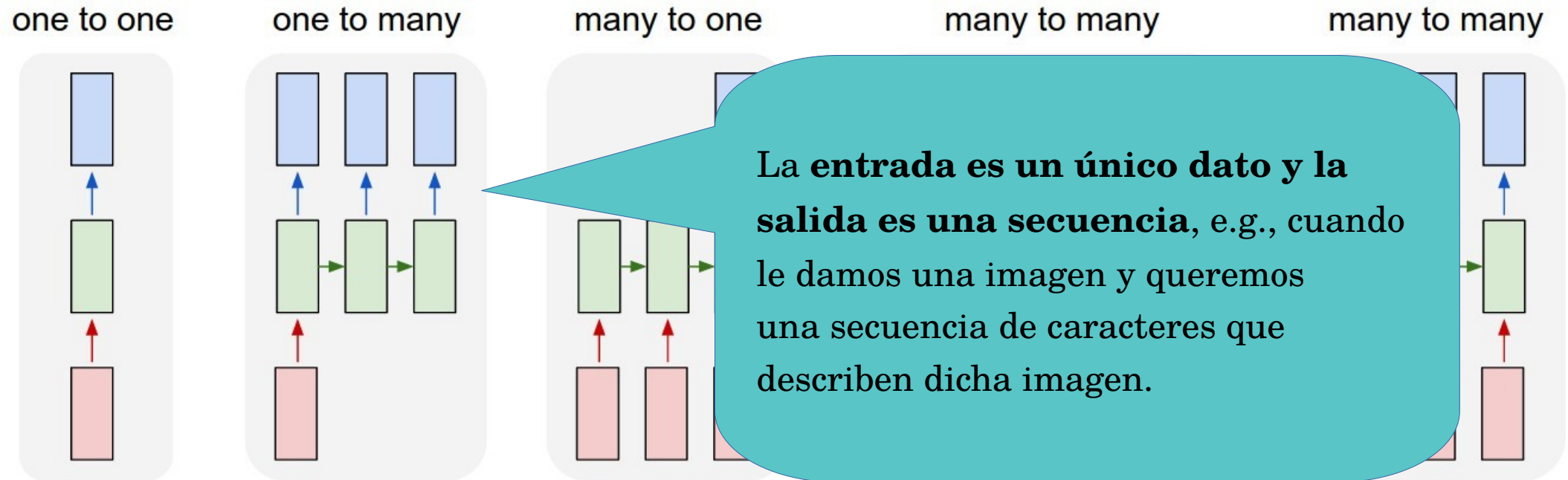
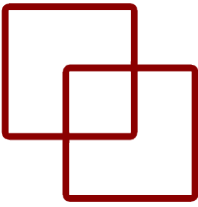
many to many



Fuente: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

1. Background

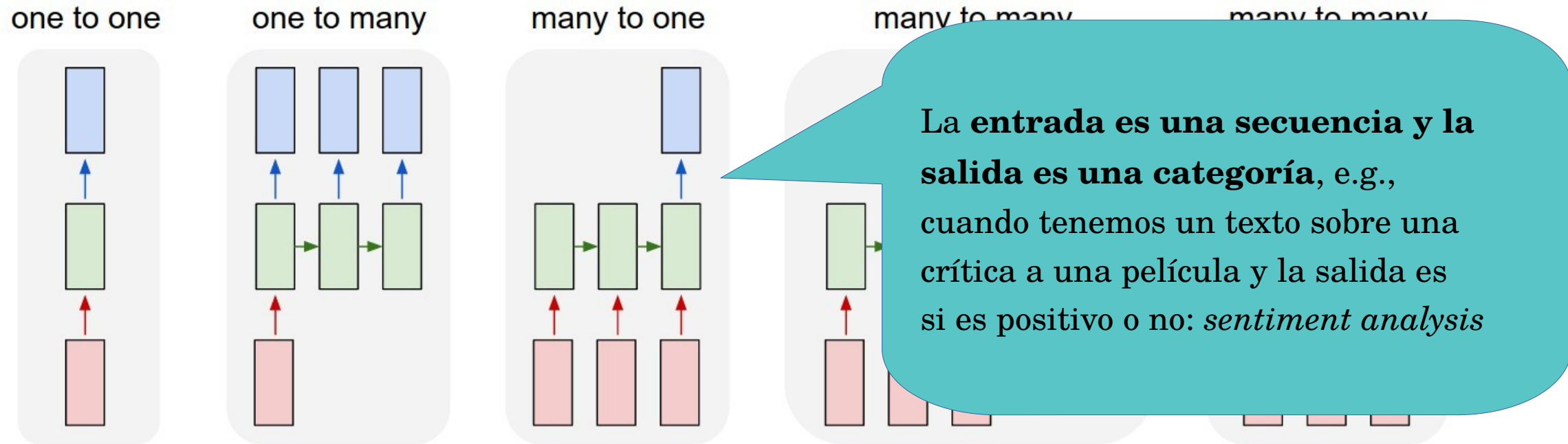
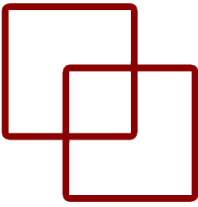
Soporte para secuencias en NN



Fuente: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

1. Background

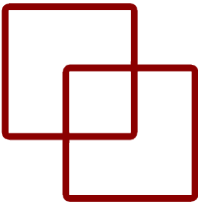
Soporte para secuencias en NN



Fuente: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

1. Background

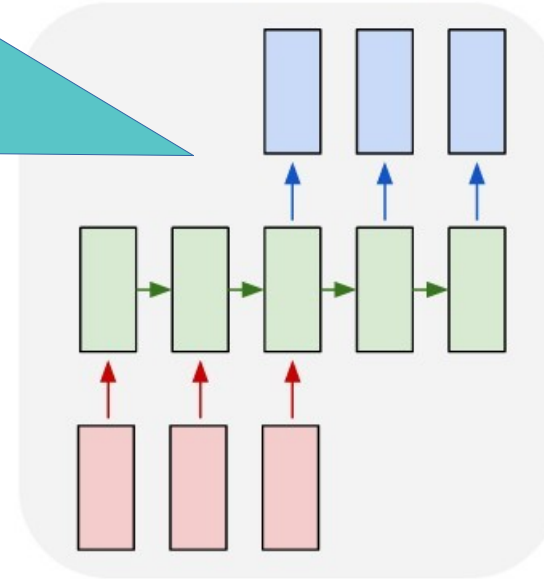
Soporte para secuencias en NN



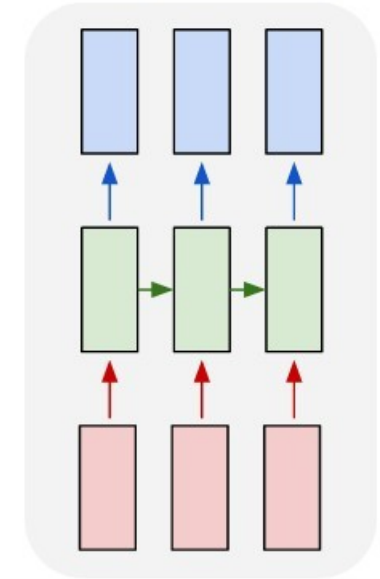
one to

Tanto la **entrada y la salida se tienen secuencias**, e.g., traductores automáticos donde se requiere tener el texto completo del español para poder traducirlo al inglés. Otro ejemplo son los conversores de texto a voz o viceversa.

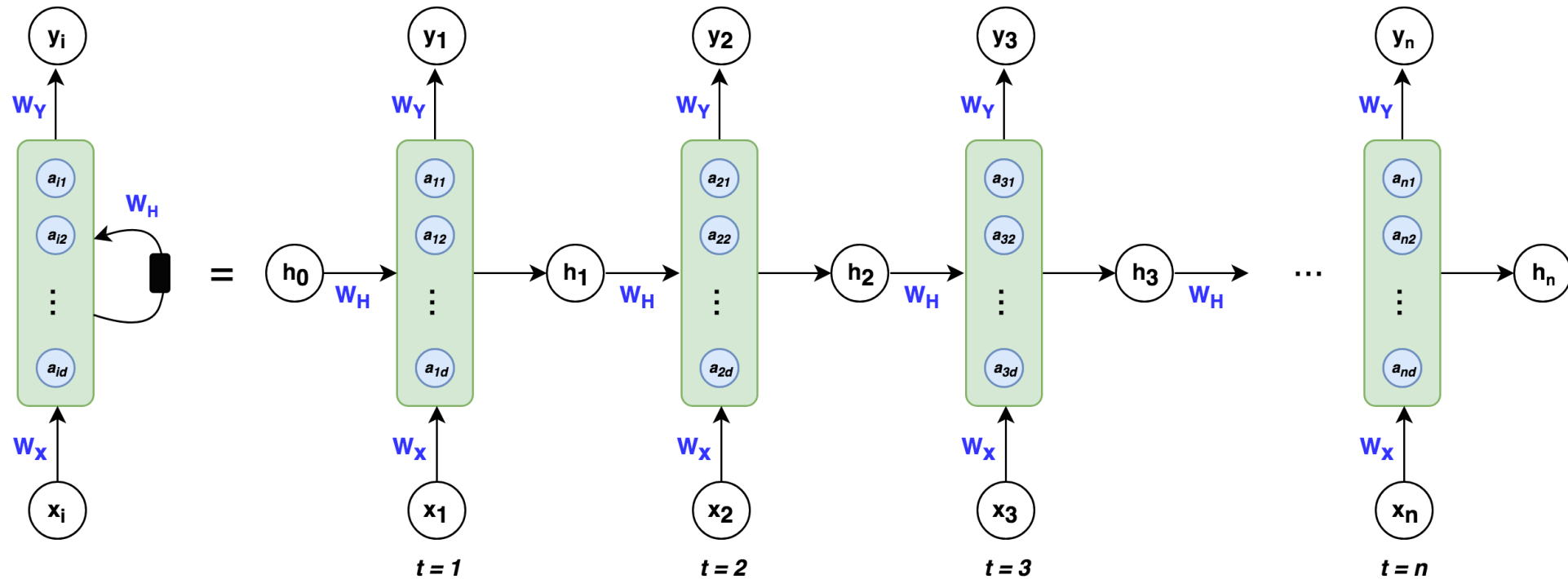
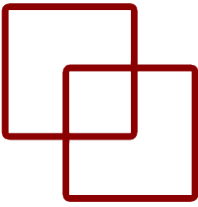
many to many



many to many



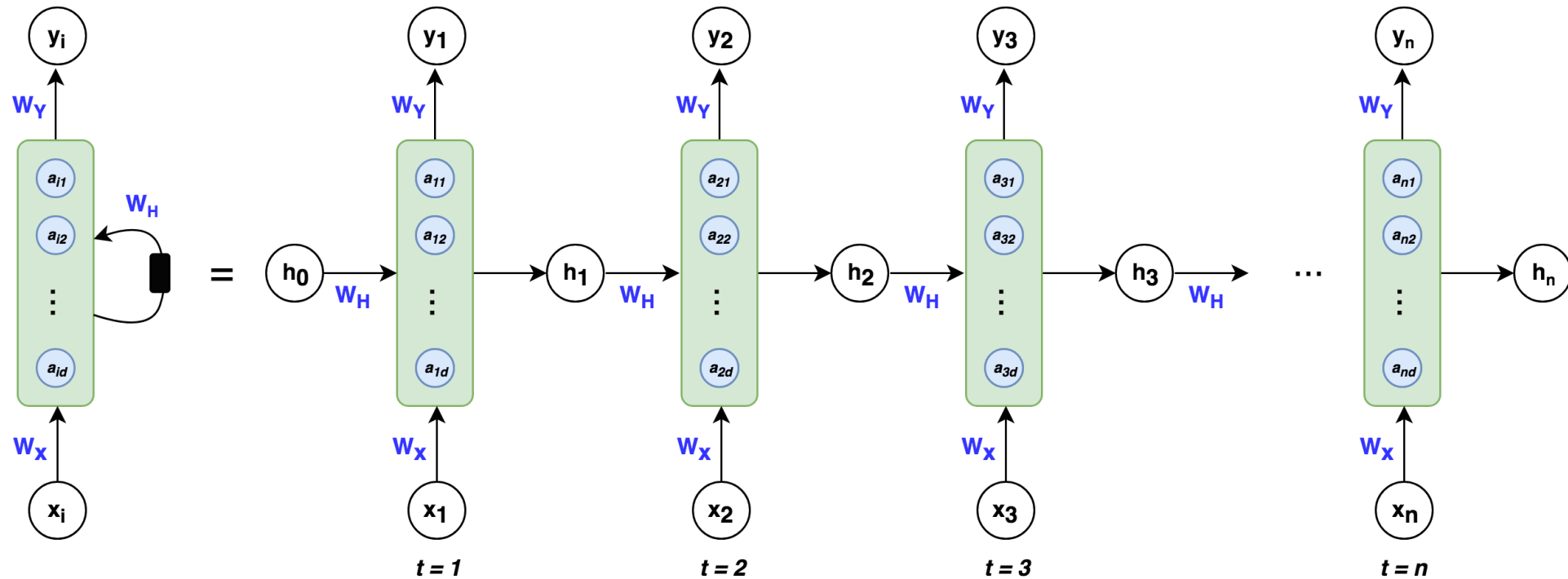
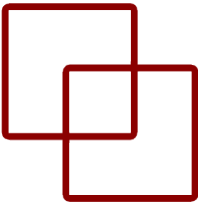
1. Background RNN



Fuente: <https://towardsdatascience.com/recurrent-neural-networks-d4642c9bc7ce?gi=a10227fa8c30>

1. Background

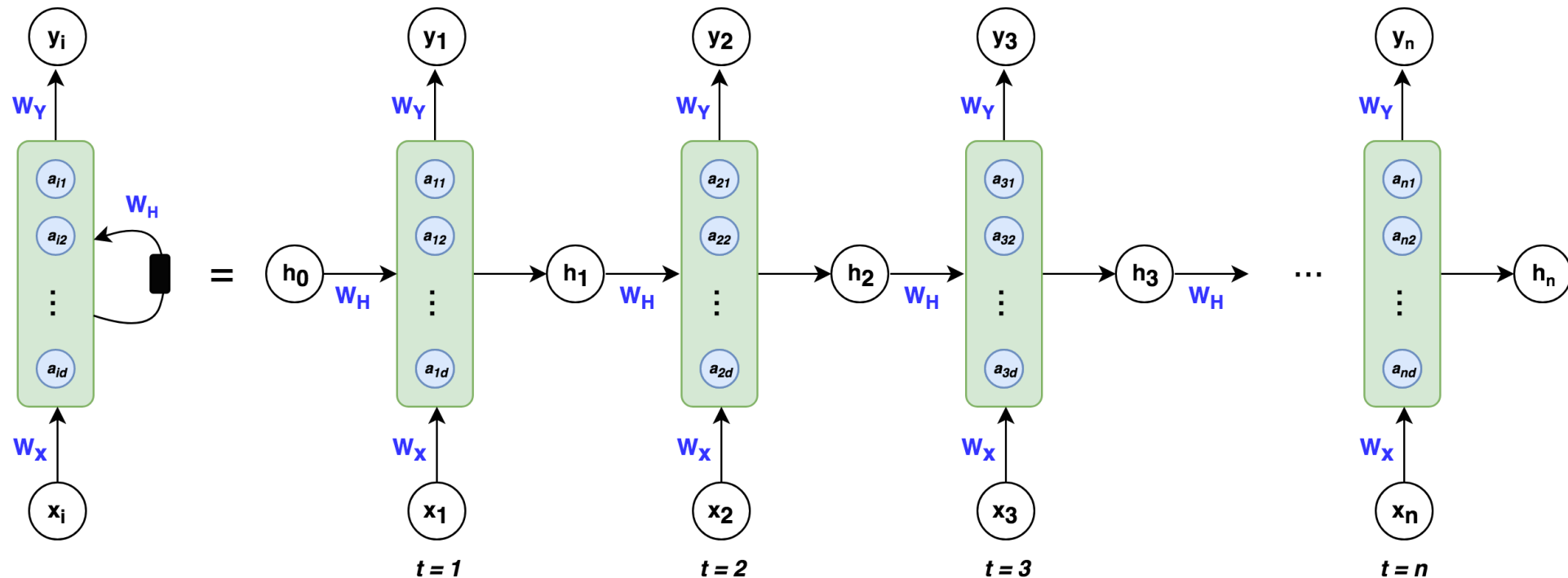
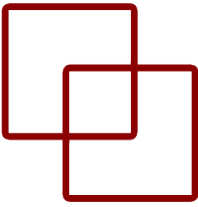
RNN - Entrenamiento



Fuente: <https://towardsdatascience.com/recurrent-neural-networks-d4642c9bc7ce?gi=a10227fa8c30>

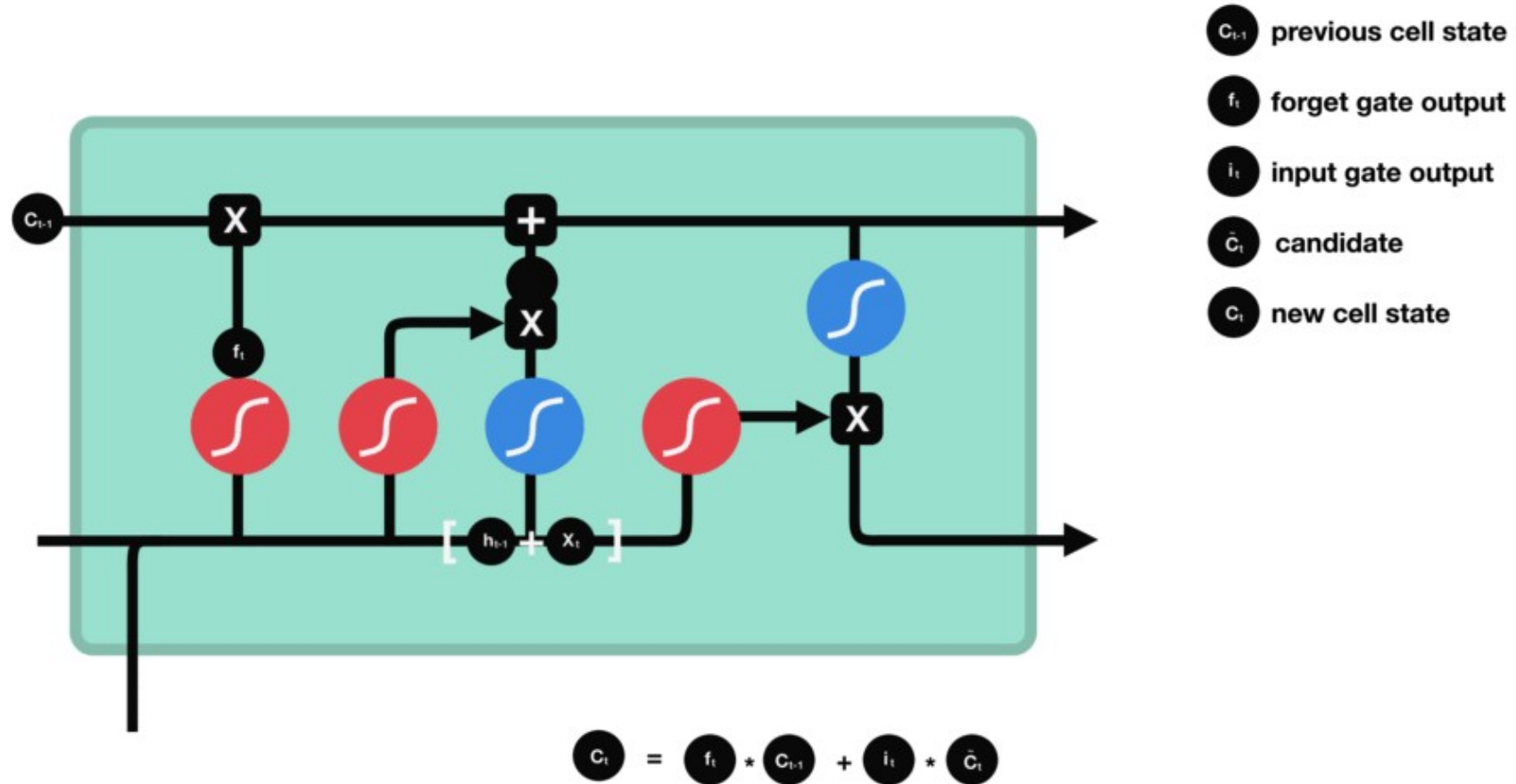
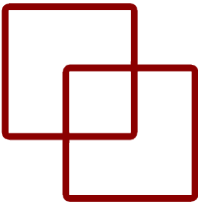
1. Background

RNN – Estabilizar el gradiente



Fuente: <https://towardsdatascience.com/recurrent-neural-networks-d4642c9bc7ce?gi=a10227fa8c30>

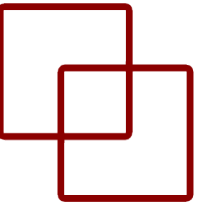
1. Background LSTM



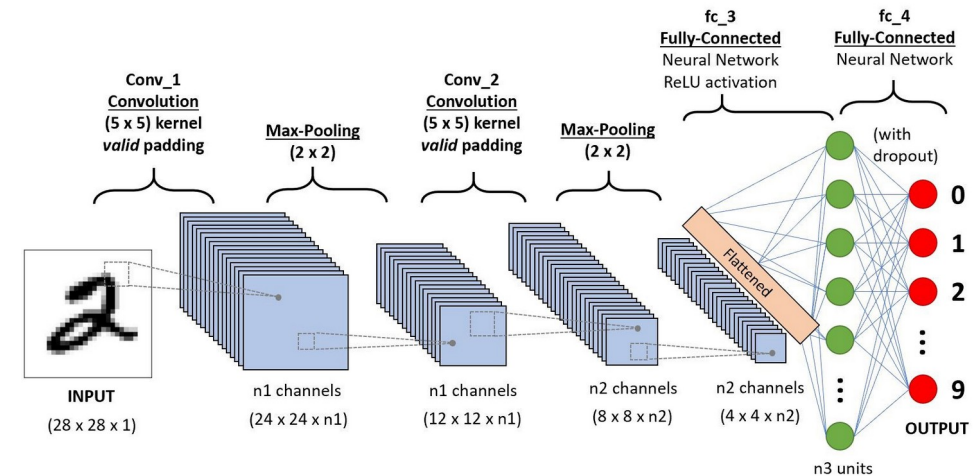
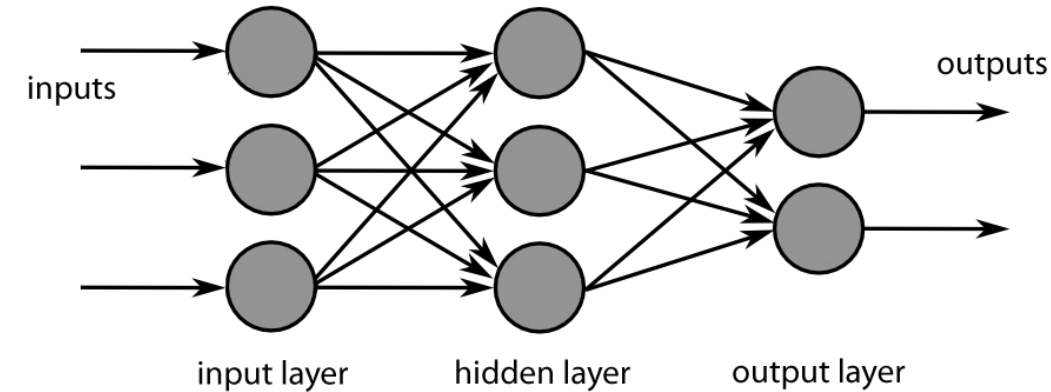


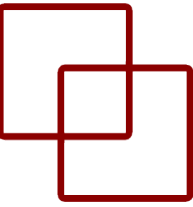
Introducción

1. Contexto (I)



- Hasta ahora hemos visto dos de los principales modelos de Deep Learning:
 - **Redes neuronales “feed-forward”**
 - Problemas de clasificación y regresión
 - **Redes Neuronales Convolucionales**
 - Principalmente Imágenes
- ¿Cómo gestionamos entradas con relación temporal entre los datos?
- ¿Y otros tipos de aprendizaje?
 - Aprendizaje por refuerzo





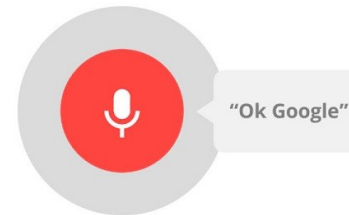
1. Contexto (I)

- Reconocimiento de voz
- Procesamiento del lenguaje natural
- Predicción de series temporales
- Reconocimiento de gestos
- Aprendizaje por refuerzo

NETFLIX

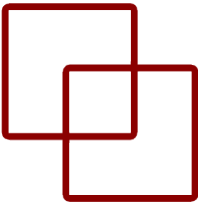


amazon alexa



.....





2. Generación de texto

at first:

tyntd-iafhatawiaoighrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrgrd t o idoe ns,smtt h ne etie h,hregtrs nigtkie,aoaenns lng

↓ train more

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

↓ train more

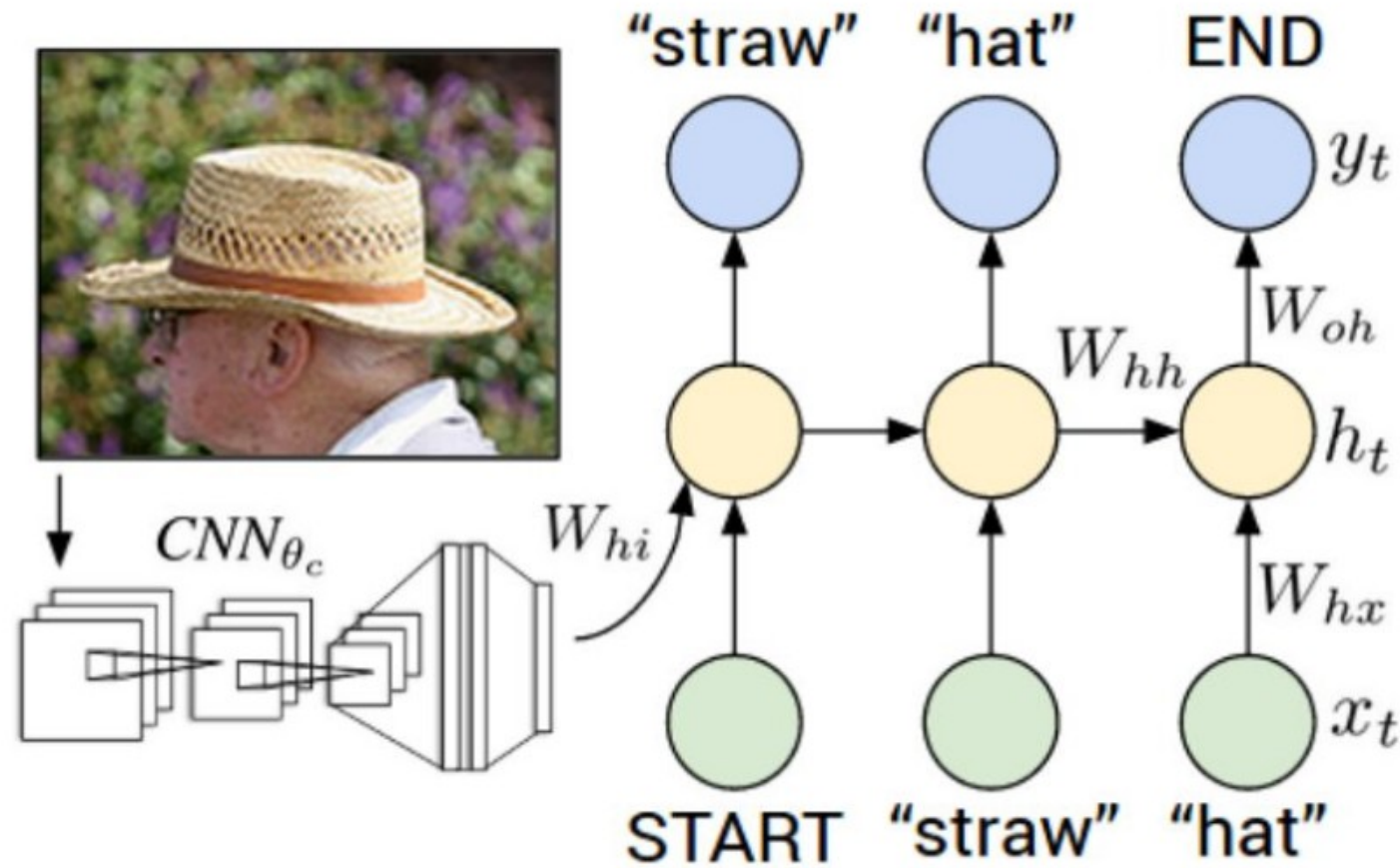
Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.

↓ train more

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftended him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

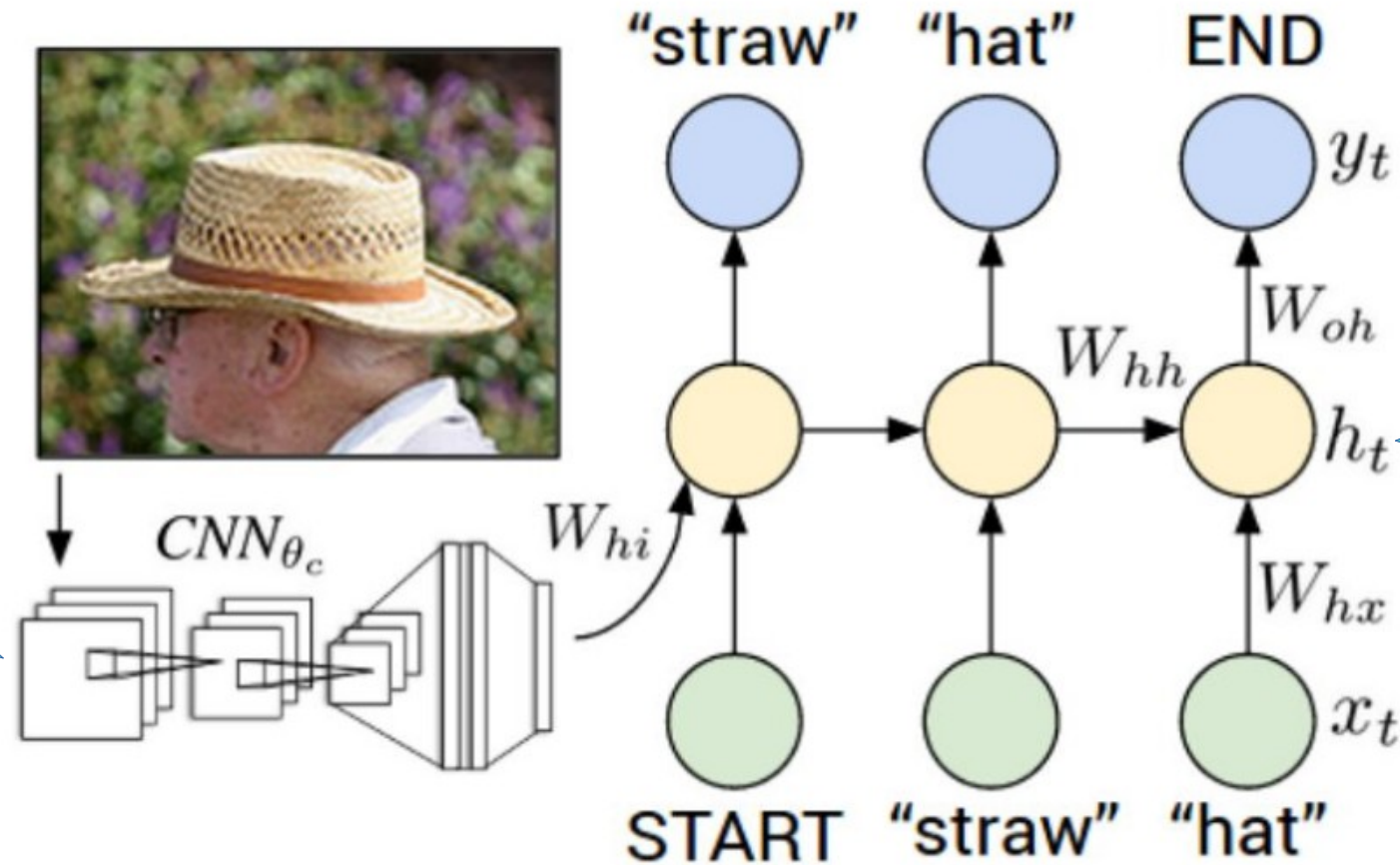
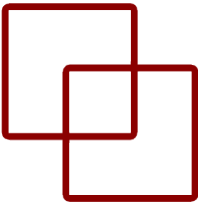
Fuente: <http://cs231n.stanford.edu/index.html>

3. Captar imagen



Fuente: Karpathy, A., & Fei-Fei, L. (2015). Deep visual-semantic alignments for generating image descriptions. In Proceedings of the IEEE conference on computer vision and pattern recognition.

3. Captar imagen

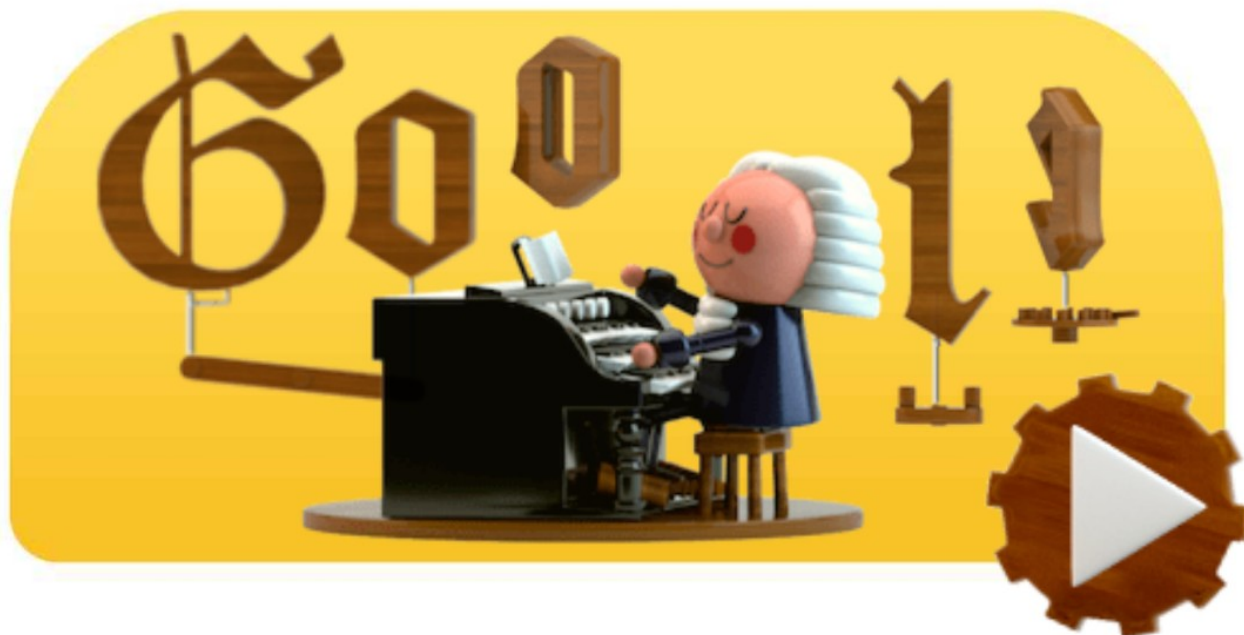


Parte CNN -
Extraer característ.

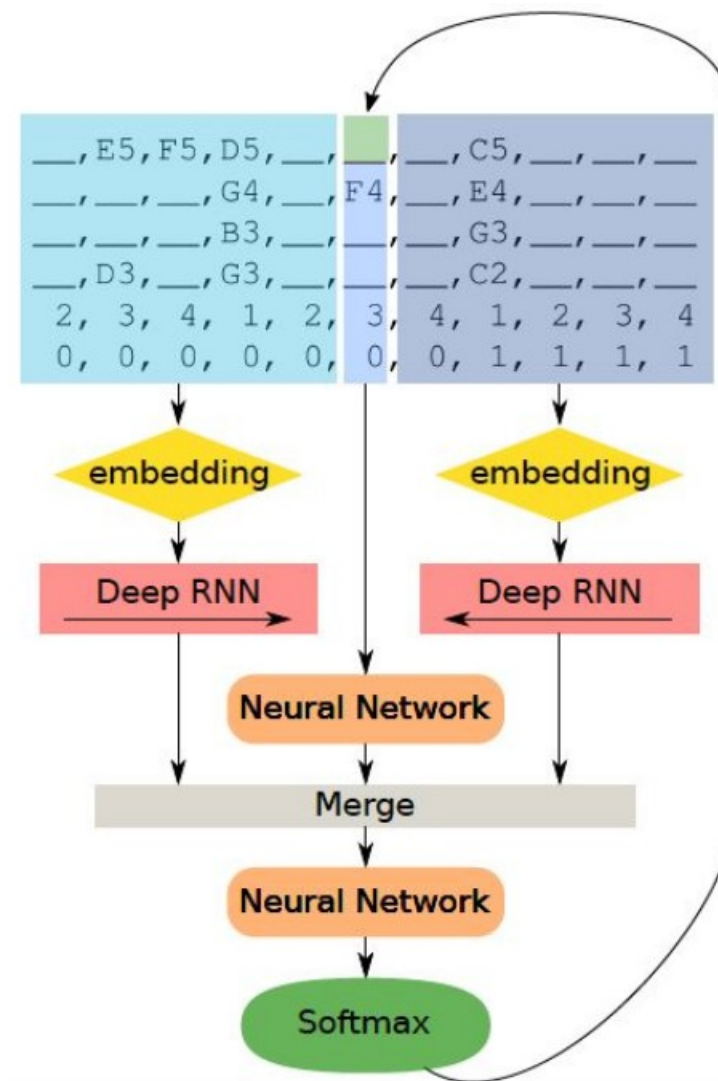
Parte RNN -
Contenidos
semántico

Fuente: Karpathy, A., & Fei-Fei, L. (2015). Deep visual-semantic alignments for generating image descriptions. In Proceedings of the IEEE conference on computer vision and pattern recognition.

4. DeepBach



Doodle del 21 de marzo de 2019
¡Funcionaba con CNNs!



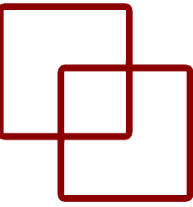
Fuente: Hadjeres, G., Pachet, F., & Nielsen, F. (2017). DeepBach: a steerable model for Bach chorales generation. In Proceedings of the 34th International Conference on Machine Learning



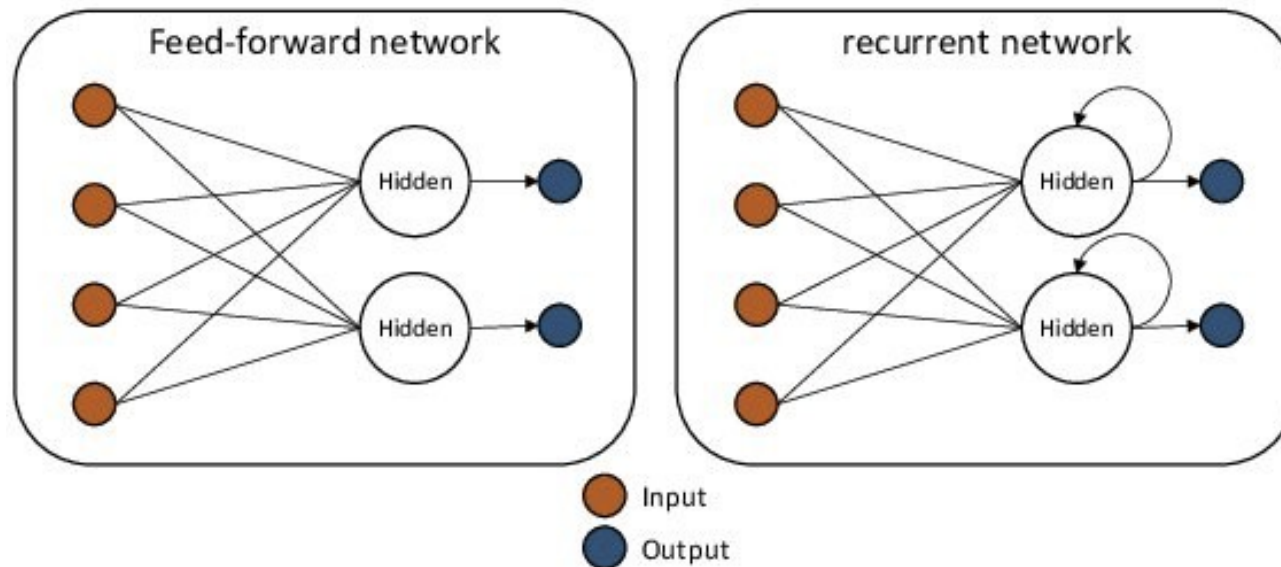
LAB CTIC  UNI

Redes Feed Forward y Recurrent Neural Networks

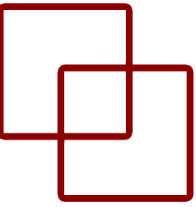
1. Feed Forward



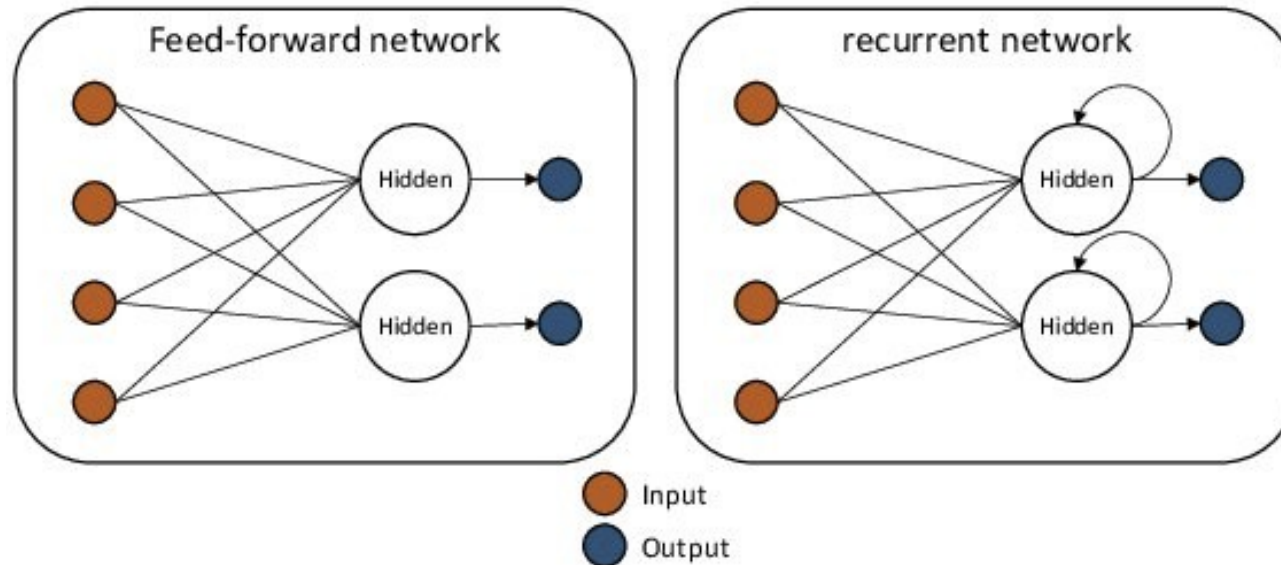
- A pesar de que se verán en la última sesión, es importante conocer la diferencia entre las dos principales **topologías** de redes
- Las llamadas **Feed Forward** se corresponden con el modelo tradicional, y se definen como **redes sin ciclos** ni **bucles** durante la clasificación, i.e., sufren de **amnesia**.
 - El flujo de datos pasa de las capas de entrada a las intermedias, y finalmente a la capa de salida donde se obtiene el resultado final

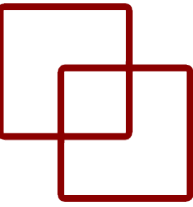


2. Recurrent Neural Networks



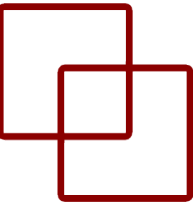
- Las redes Recurrent Neural Networks (RNNs), por el contrario, permiten la **aparición de bucles** en su arquitectura.
- Estos bucles tienen diferentes implicaciones, como la capacidad de aprovechar la temporalidad de una secuencia de entrada a través de un **estado**
 - Podemos decir que una red puede procesar una imagen en el **instante t** , y utilizar este último estado para procesar la siguiente imagen capturada en el **instante $t+1$** .





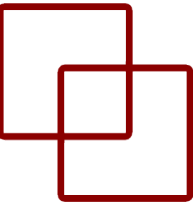
3. Características de FFN y RNN

- Los resultados usando RNNs han sido especialmente buenos hasta el momento en el reconocimiento del audio
 - Línea de investigación que permanece abierta



3. Características de FFN y RNN

- Los resultados usando RNNs han sido especialmente buenos hasta el momento en el reconocimiento del audio
 - Línea de investigación que permanece abierta
- Sin embargo, son **difíciles de entrenar** y en muchos casos no resulta práctico su uso ante problemas que puedan resolverse con redes Feed Forward.
 - Recomendable únicamente para problemas que requieran esa potencia.



3. Características de FFN y RNN

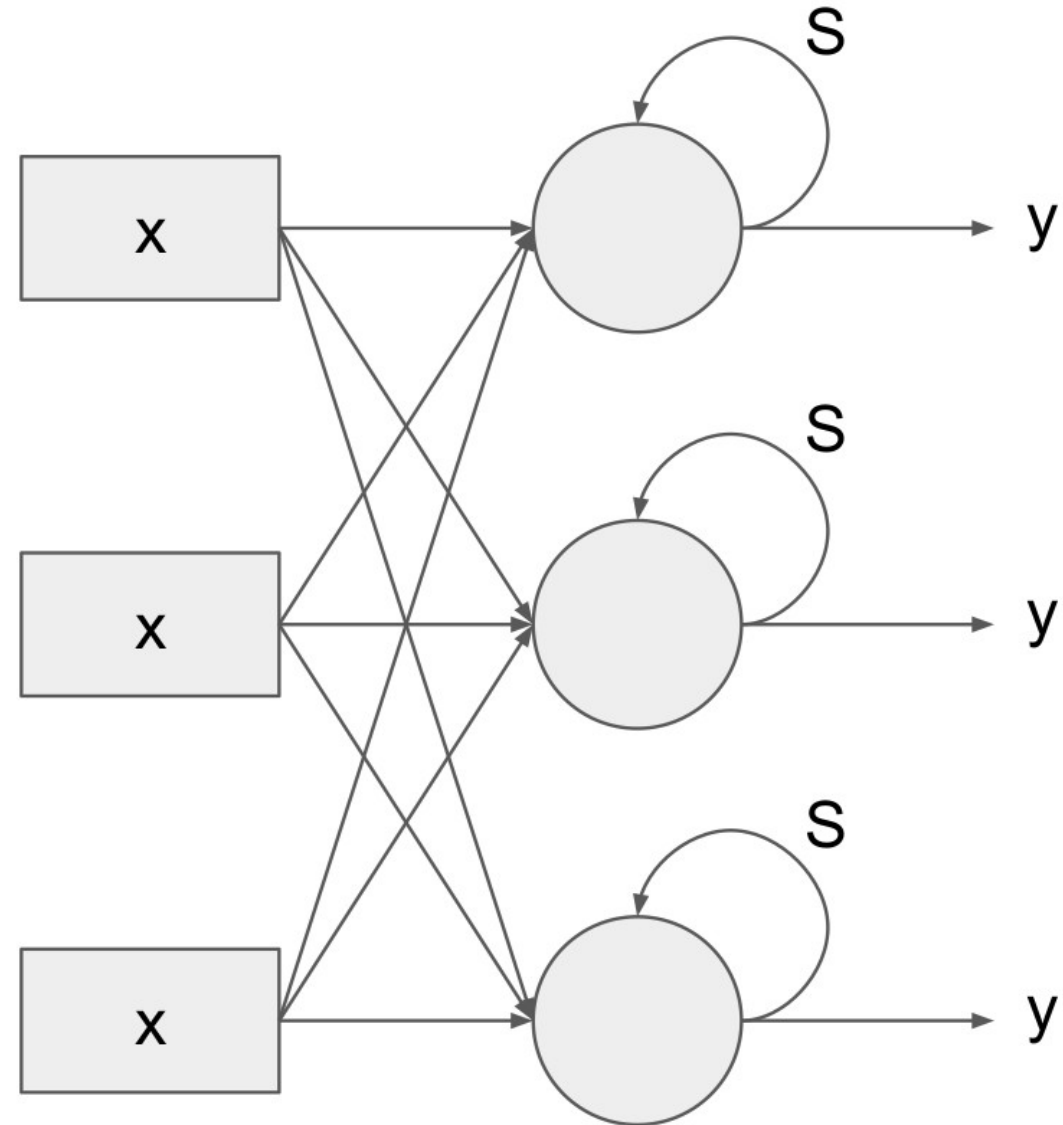
- Los resultados usando RNNs han sido especialmente buenos hasta el momento en el reconocimiento del audio
 - Línea de investigación que permanece abierta
- Sin embargo, son **difíciles de entrenar** y en muchos casos no resulta práctico su uso ante problemas que puedan resolverse con redes Feed Forward.
 - Recomendable únicamente para problemas que requieran esa potencia.
- Existe un tercer tipo de red con enlaces no dirigidos llamadas **Symmetric o Restricted Boltzmann Machines**.



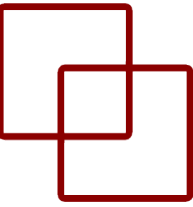
Redes Neuronales Recurrentes

1. Contexto

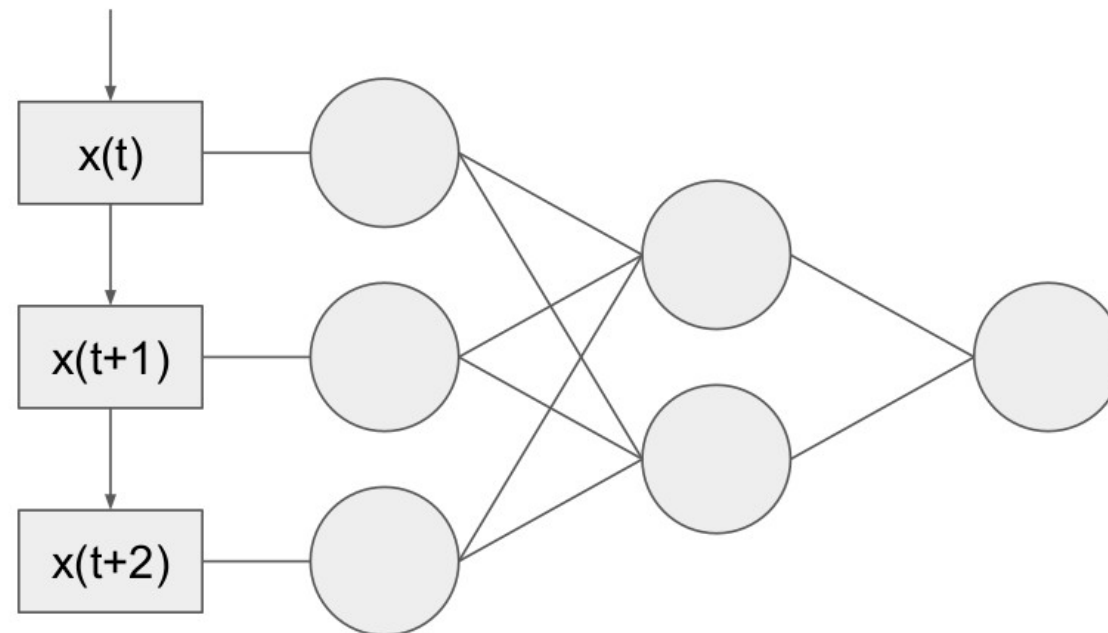
- Considera dependencias temporales.
- La salida de la red depende de la entrada actual y de entradas anteriores
 - Memoria



2. Historia (I)

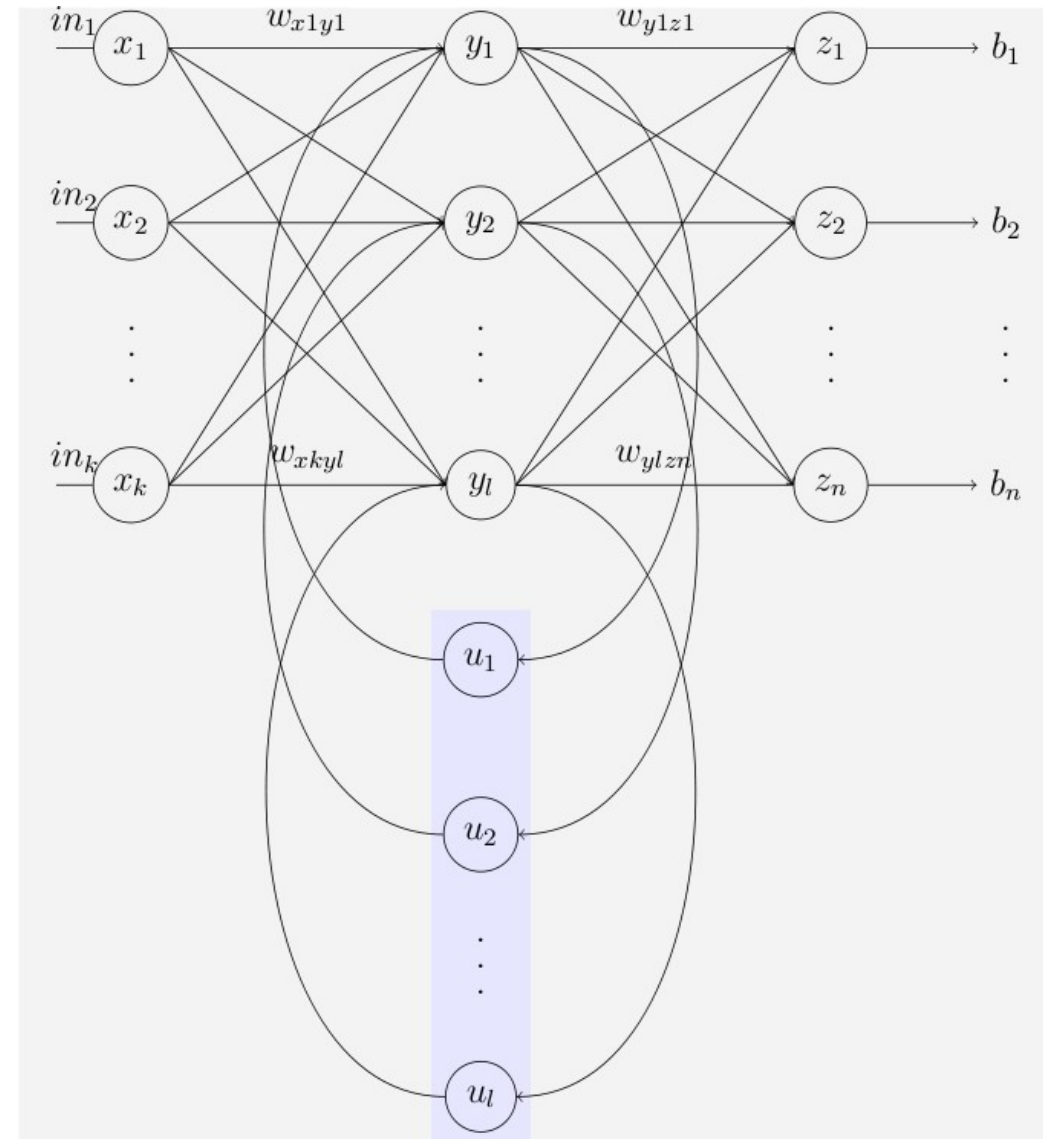


- 1989 - **Time Delay Neural Network (TDNN)**
- Entradas anteriores se añaden a la ventana actual.
 - Están limitadas al tamaño de ventana de la entrada.
 - Tenemos una entrada de tamaño fijo y no variable según el contexto



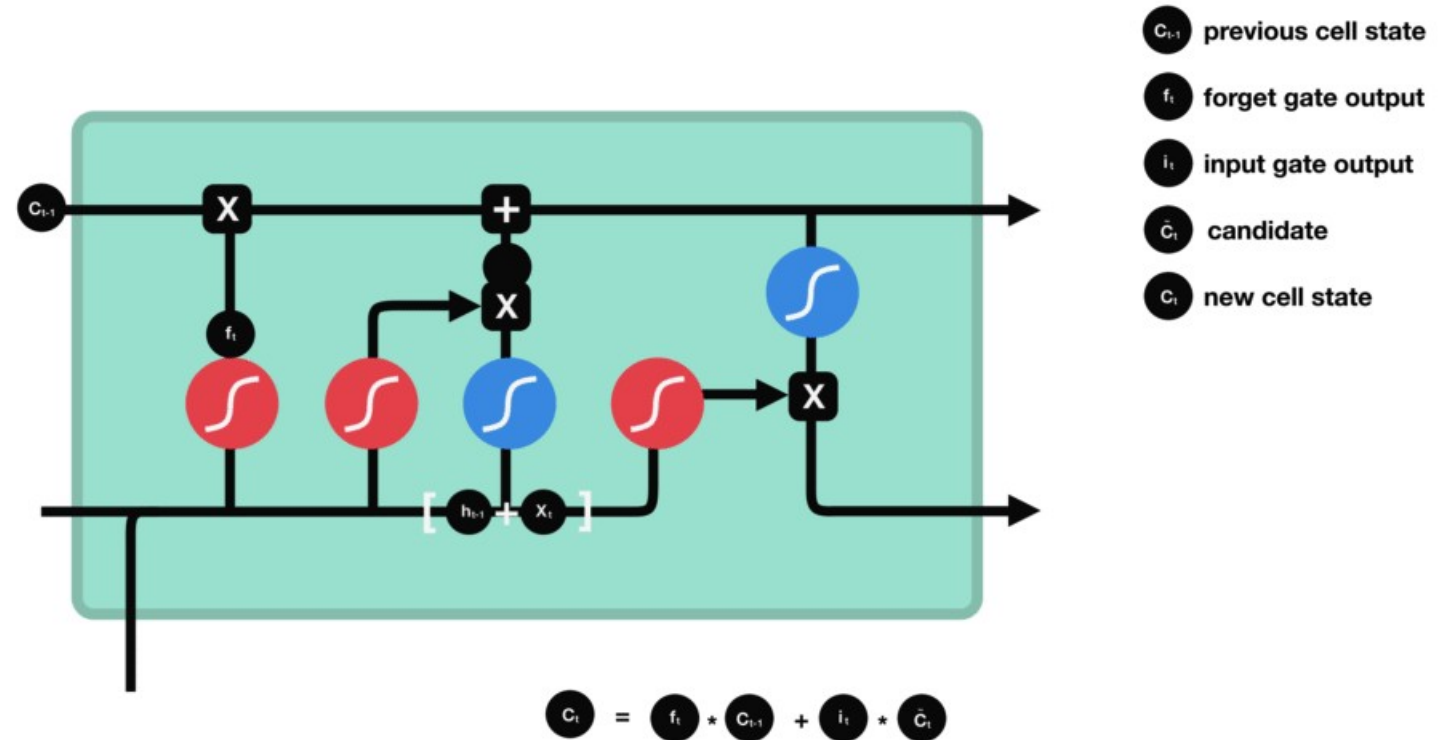
2. Historia (II)

- 1990 - **Elman Networks**
- Modelo básico de 3 capas con unidades de contexto (memoria).
- Simple RNNs.



2. Historia (III)

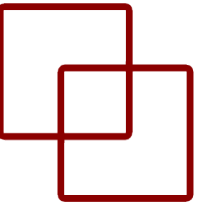
- Mediados de los 90 - **Long Short-Term Memory (LSTM)**
- Surgen para solucionar el problema del *desvanecimiento del gradiente*.



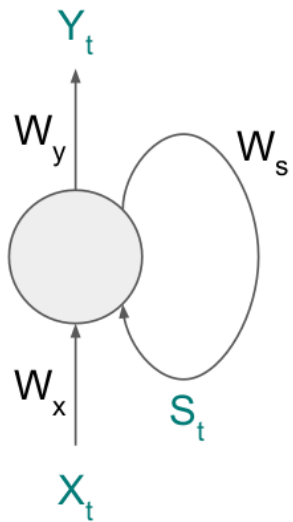
Fuente:

<https://becominghuman.ai/long-short-term-memory-part-1-3caca9889bbc>

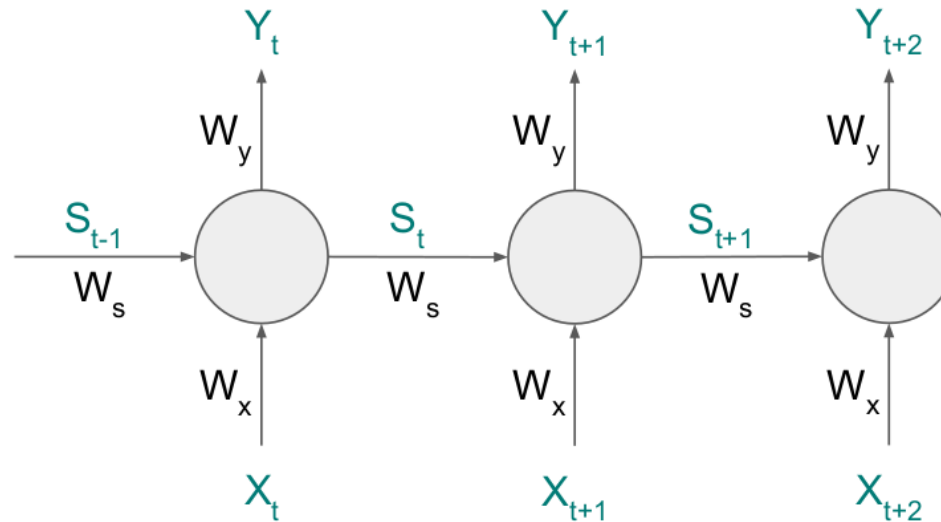
3. Funcionamiento (I)



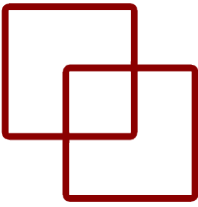
- Un mismo modelo de RNN se pueden representar de forma **plegada o desplegada**.
 - Desplegado “en tiempo”.



Modelo plegado

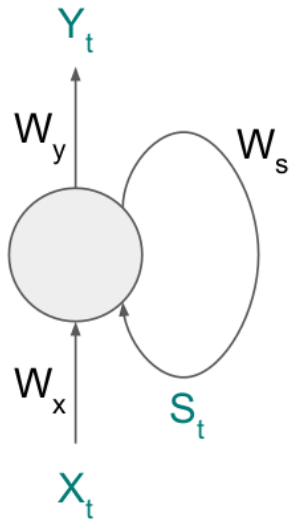


Modelo desplegado

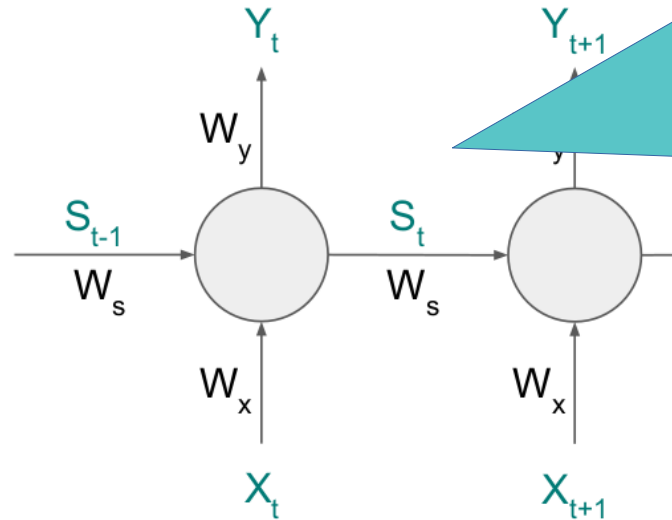


3. Funcionamiento (I)

- Un mismo modelo de RNN se pueden representar de forma **plegada o desplegada**.
 - Desplegado “en tiempo”.



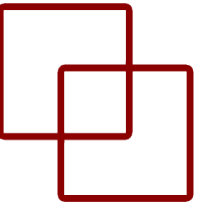
Modelo plegado



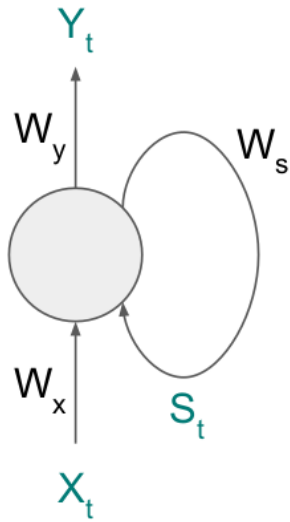
Modelo desplegado

Podemos observar que el estado actual viene dado de la activación anterior (S_{t-1}), el dato actual (X_t); mientras la salida es la predicción actual (Y_t), la activación actual (S_t) y los pesos (W)

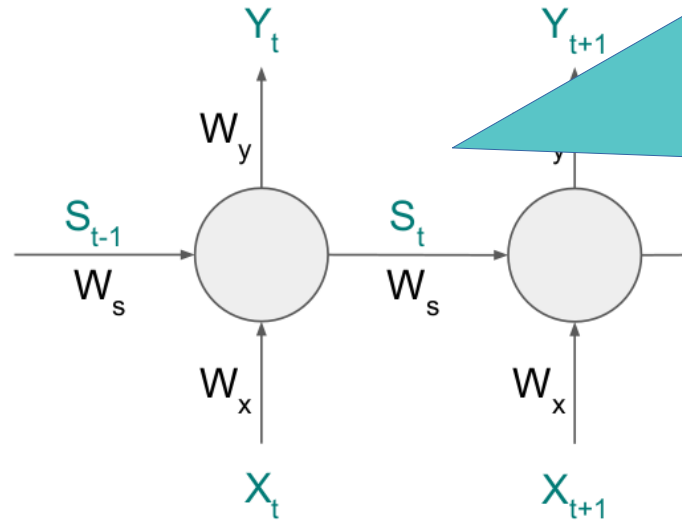
3. Funcionamiento (I)



- Un mismo modelo de RNN se pueden representar de forma **plegada o desplegada**.
 - Desplegado “en tiempo”.



Modelo plegado

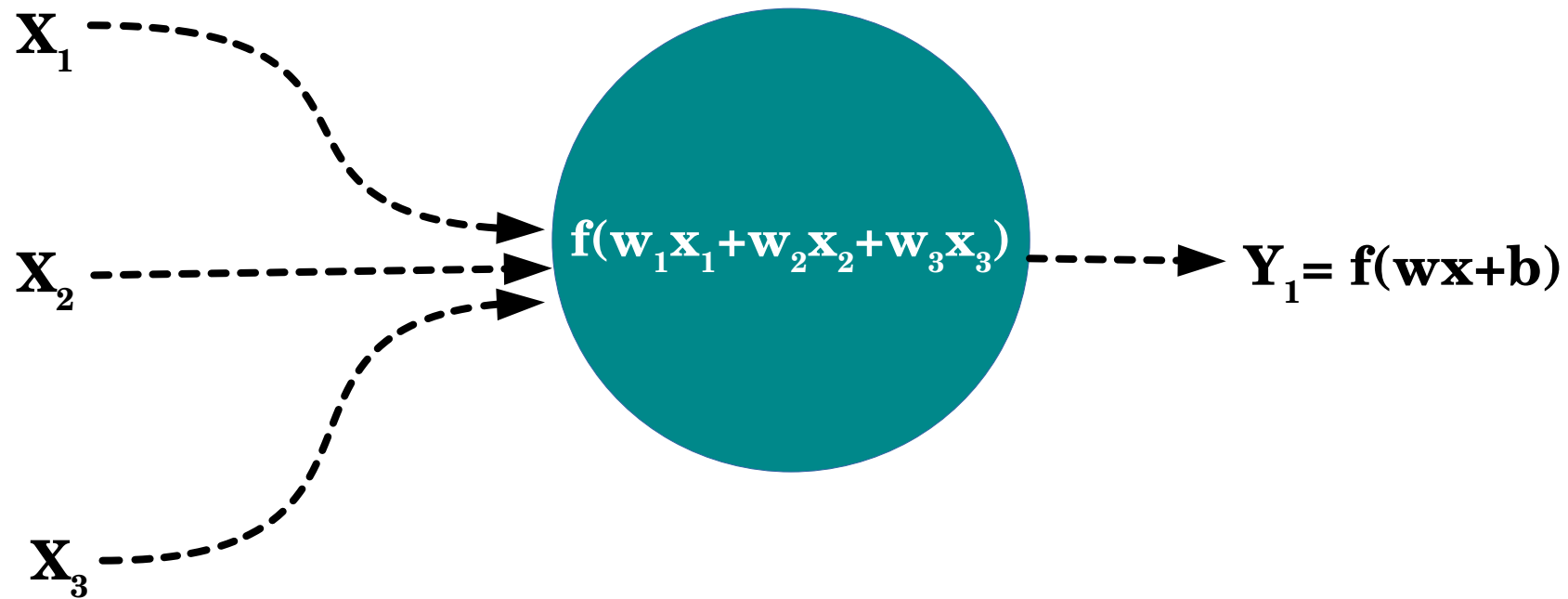
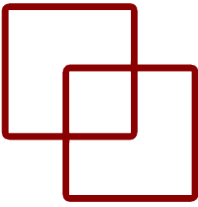


Modelo desplegado

La activación actual (S_t), llamadas estado oculto, son las que permiten preservar y compartir la información entre un estado y otro

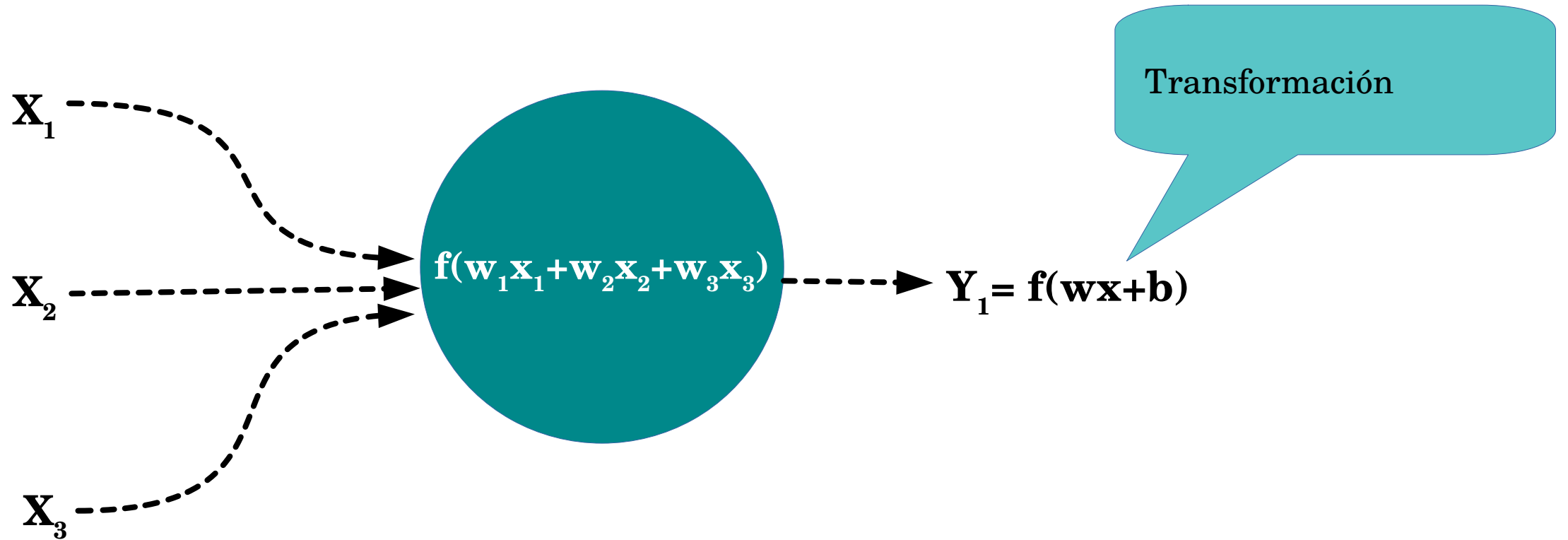
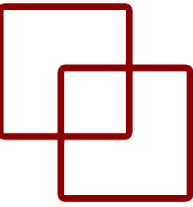
3. Funcionamiento (II)

La neurona tradicional



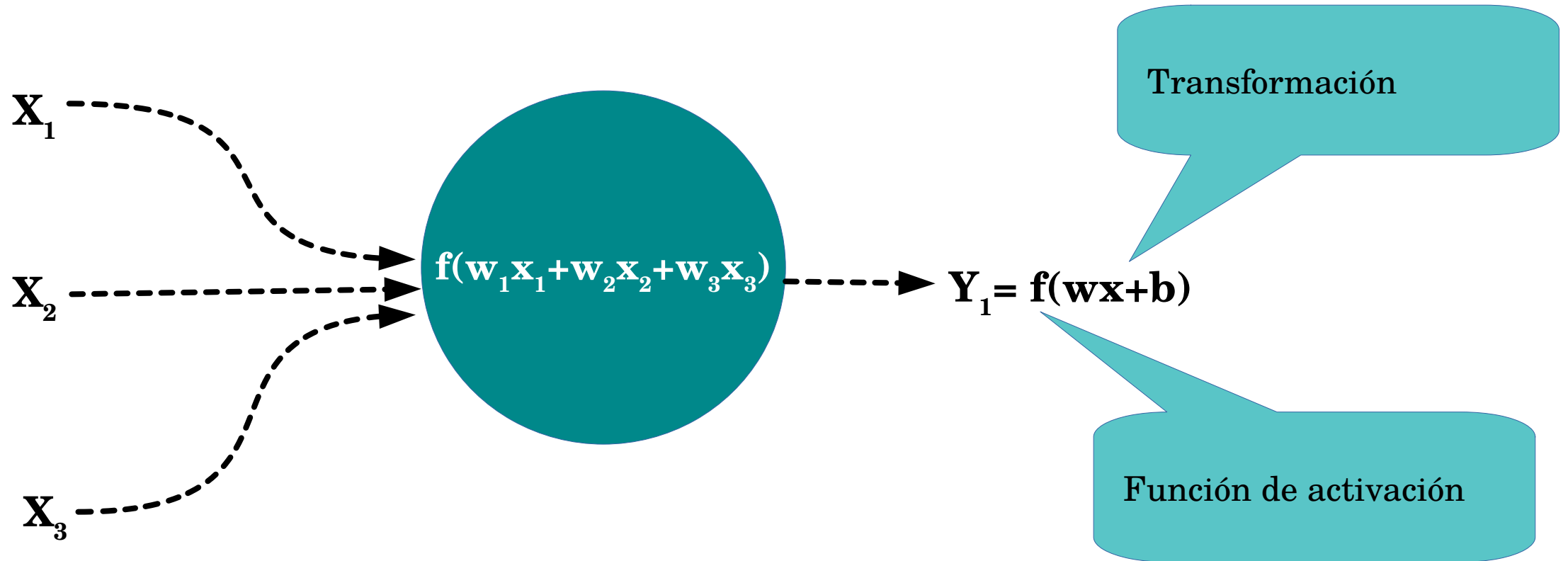
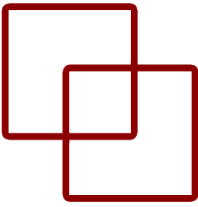
3. Funcionamiento (II)

La neurona tradicional



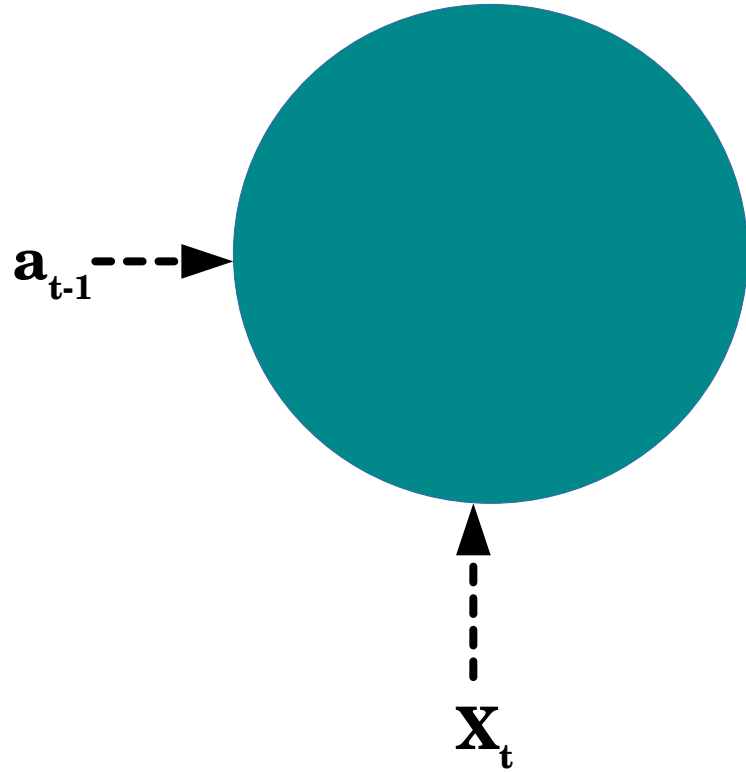
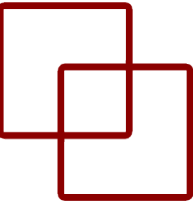
3. Funcionamiento (II)

La neurona tradicional



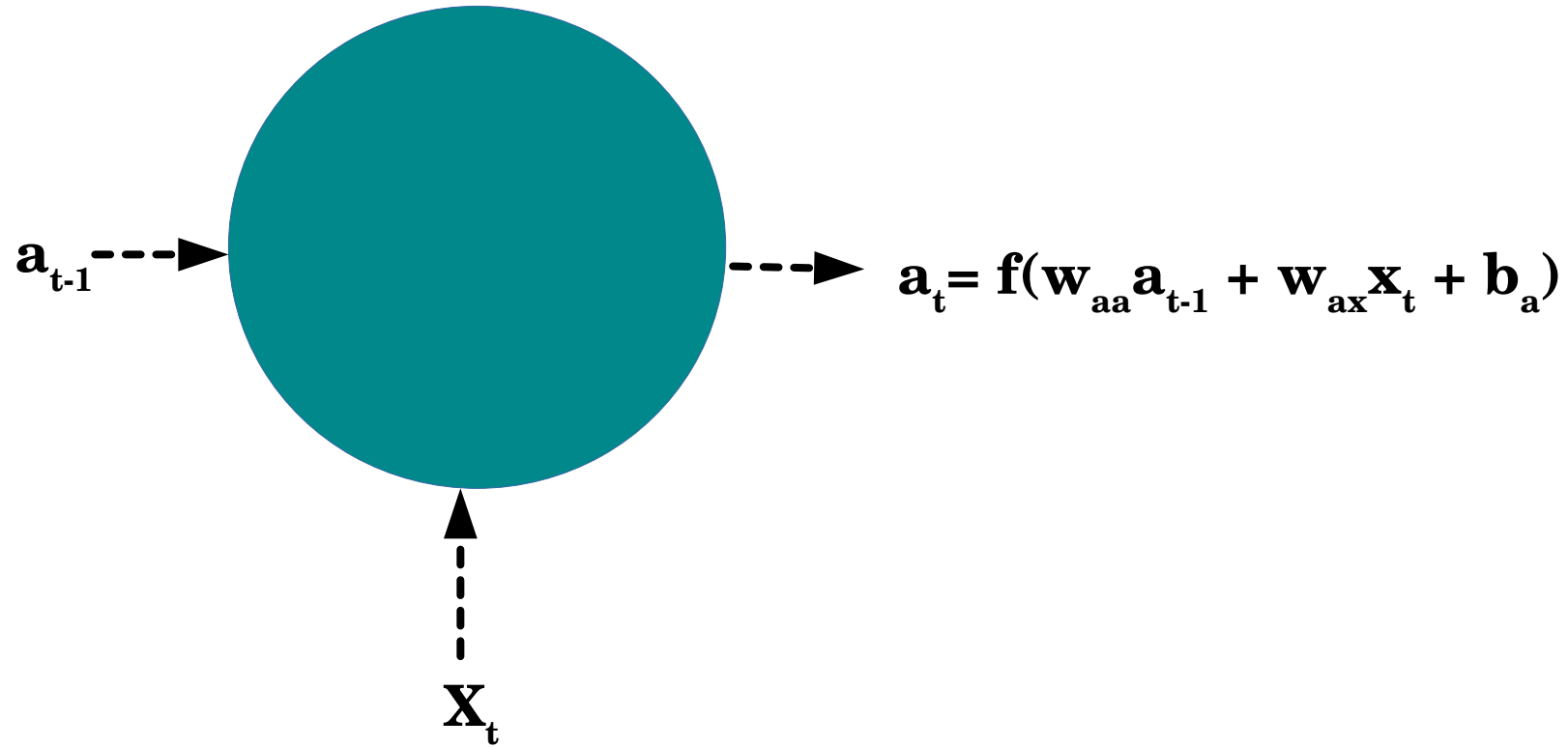
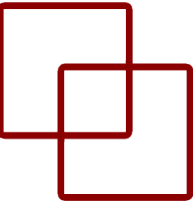
3. Funcionamiento (II)

RNN



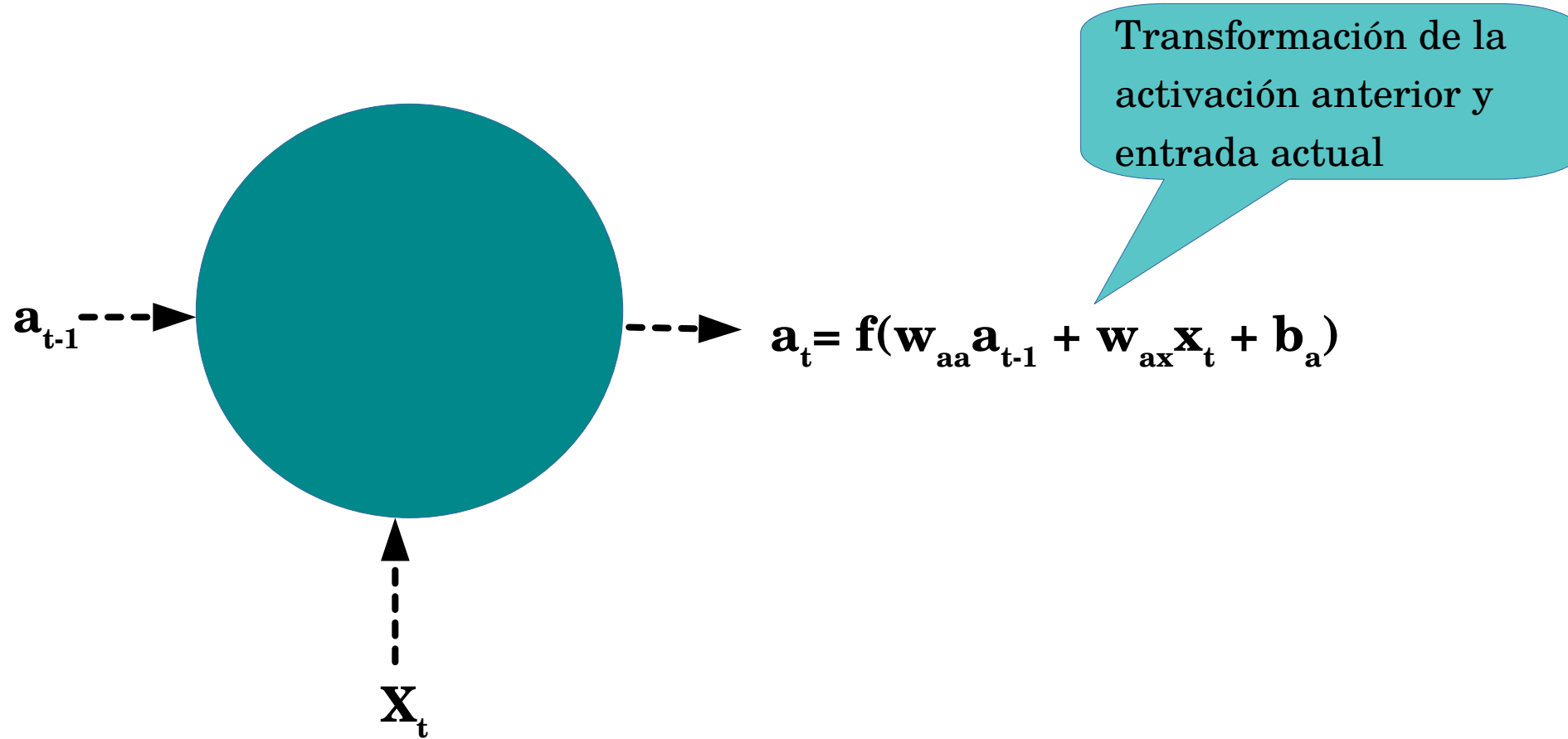
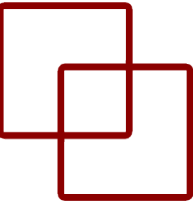
3. Funcionamiento (II)

RNN



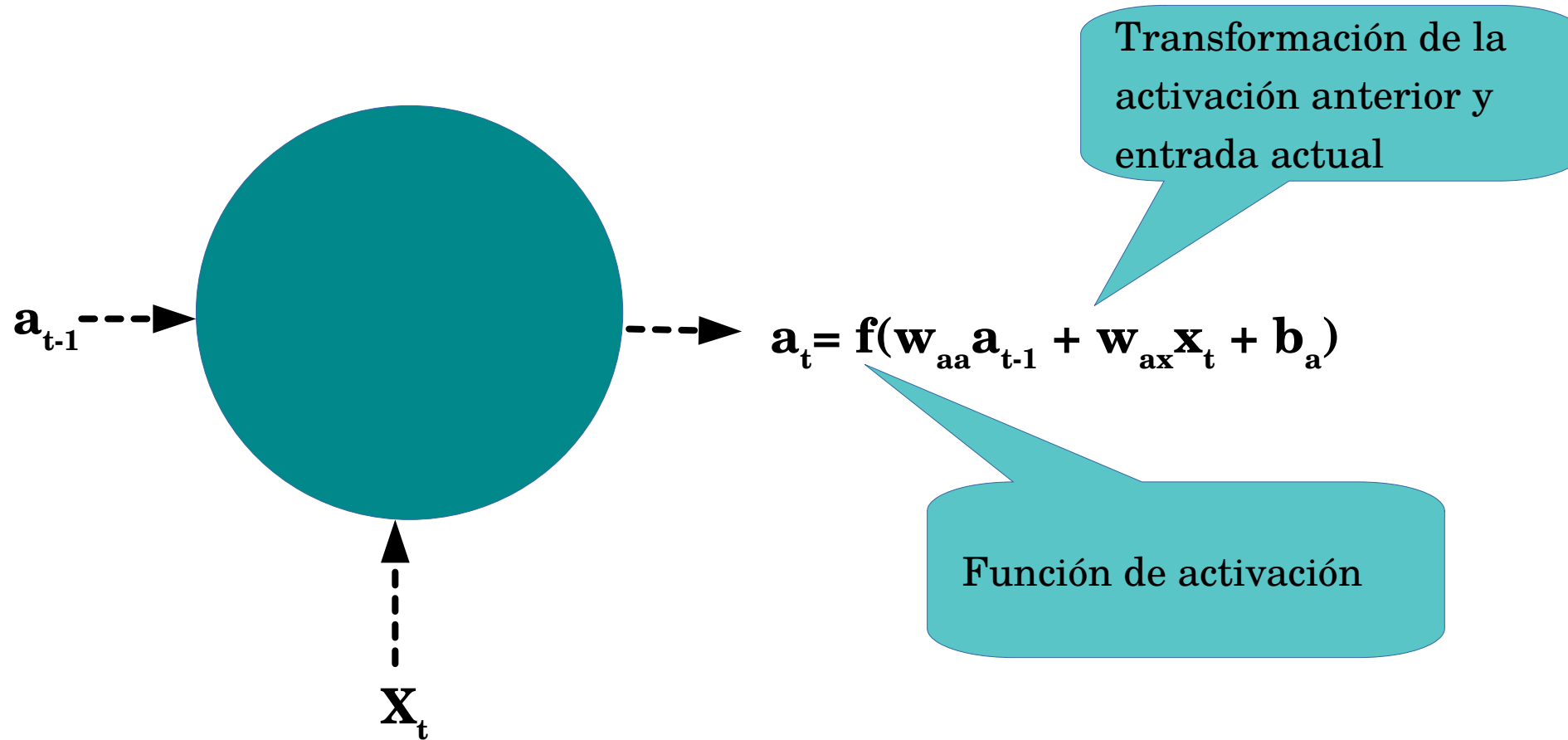
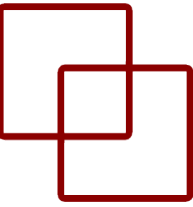
3. Funcionamiento (II)

RNN



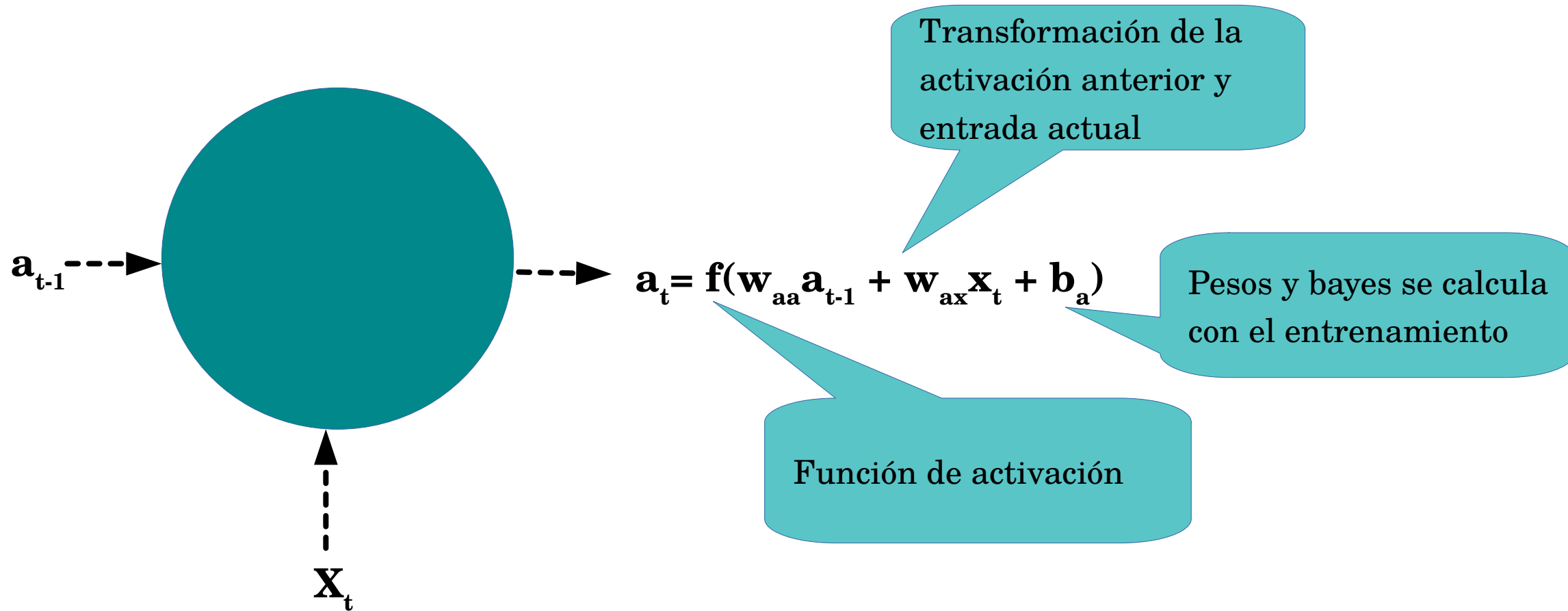
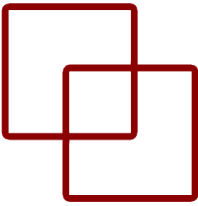
3. Funcionamiento (II)

RNN



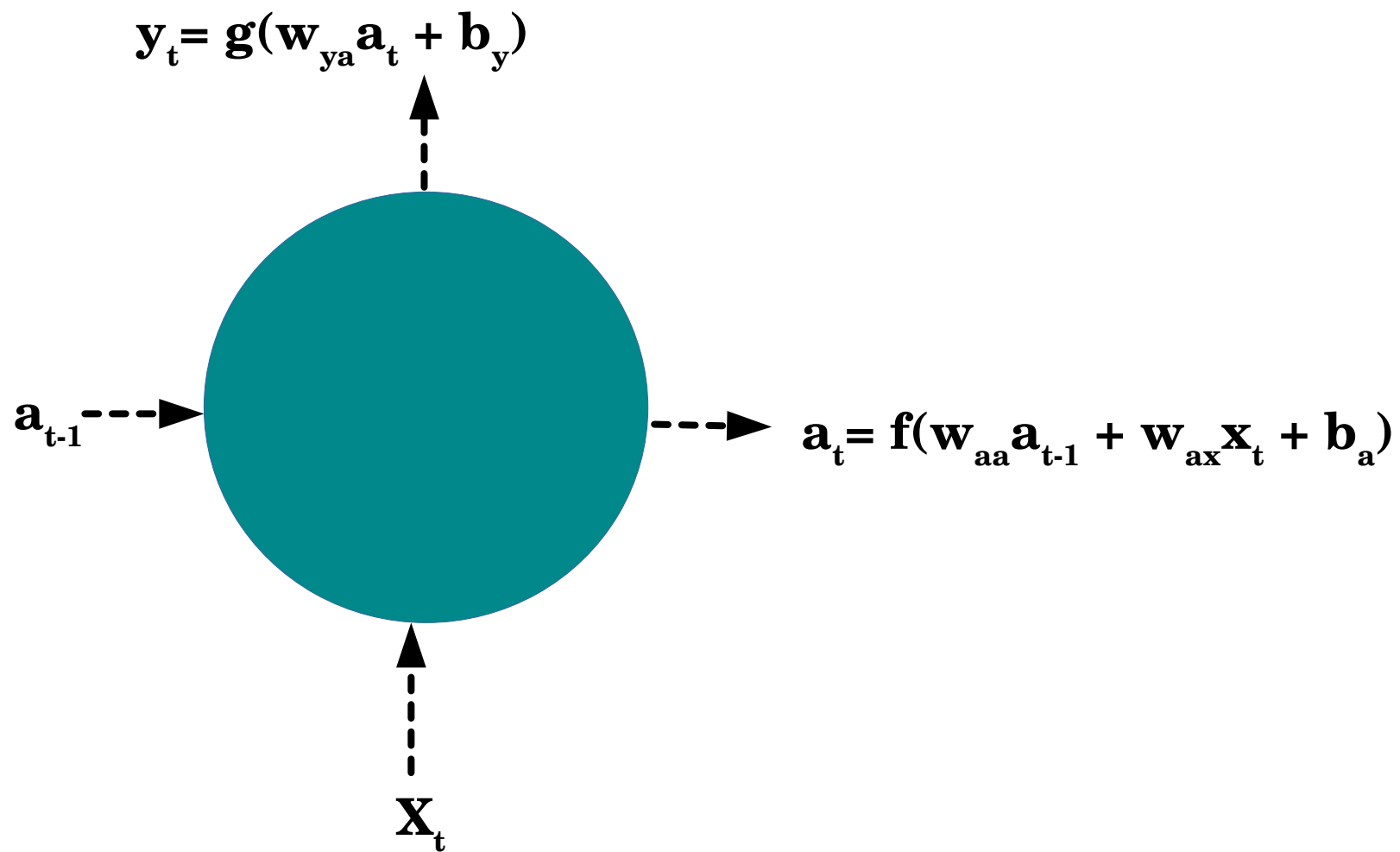
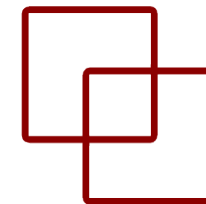
3. Funcionamiento (II)

RNN



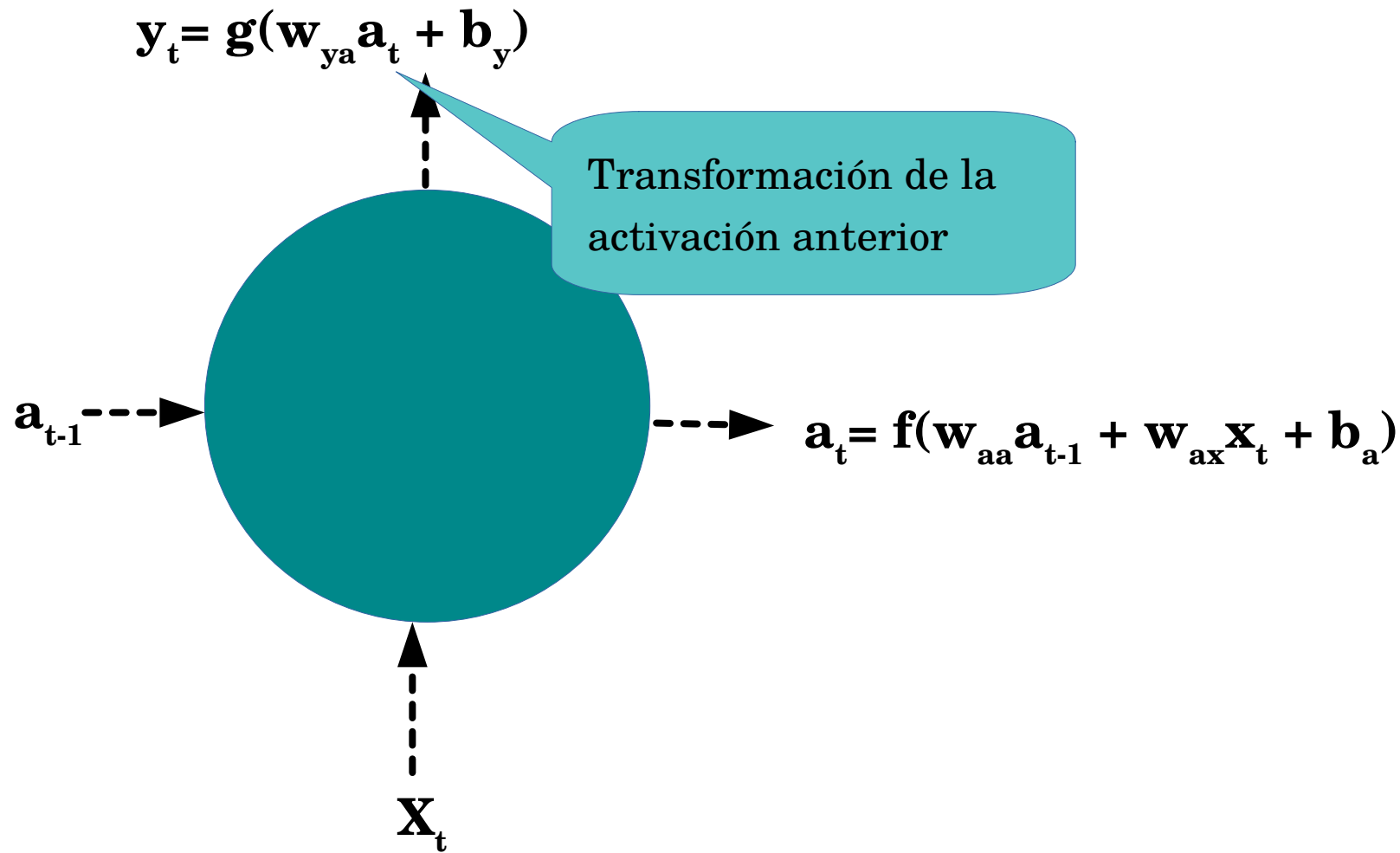
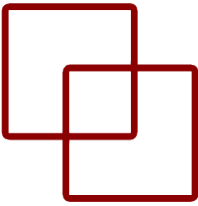
3. Funcionamiento (II)

RNN



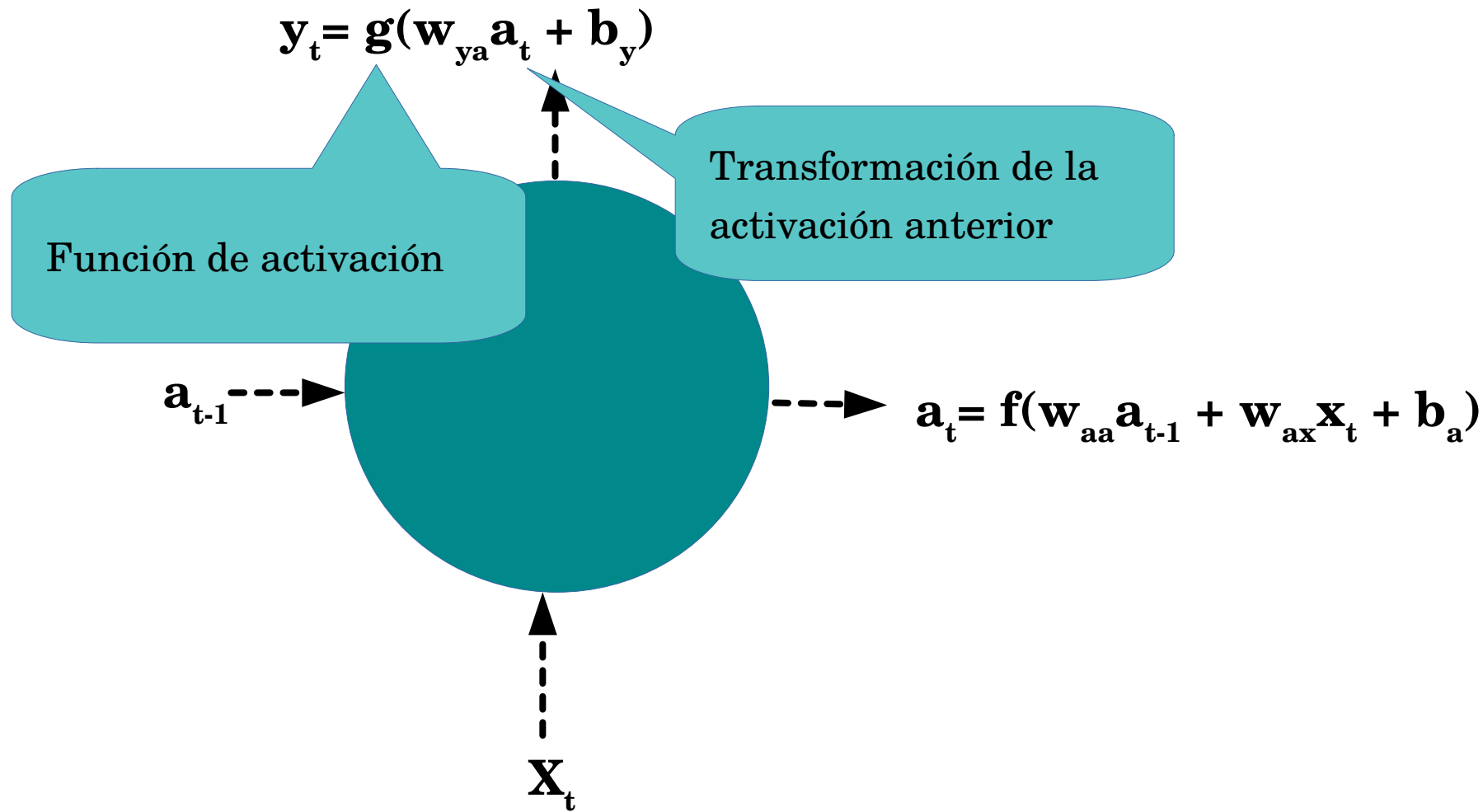
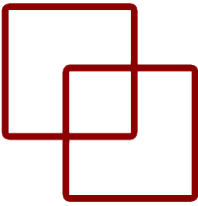
3. Funcionamiento (II)

RNN



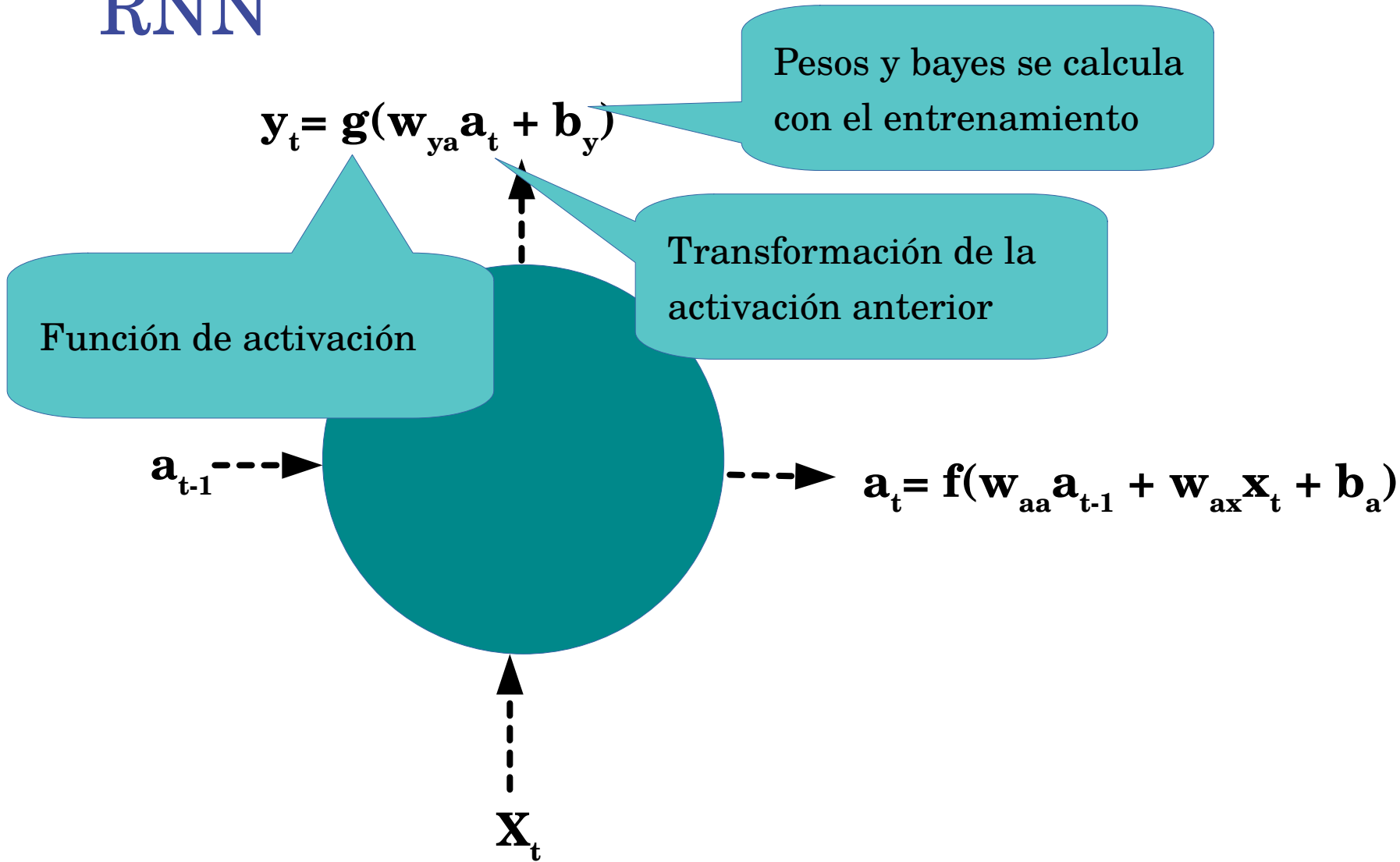
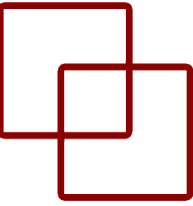
3. Funcionamiento (II)

RNN



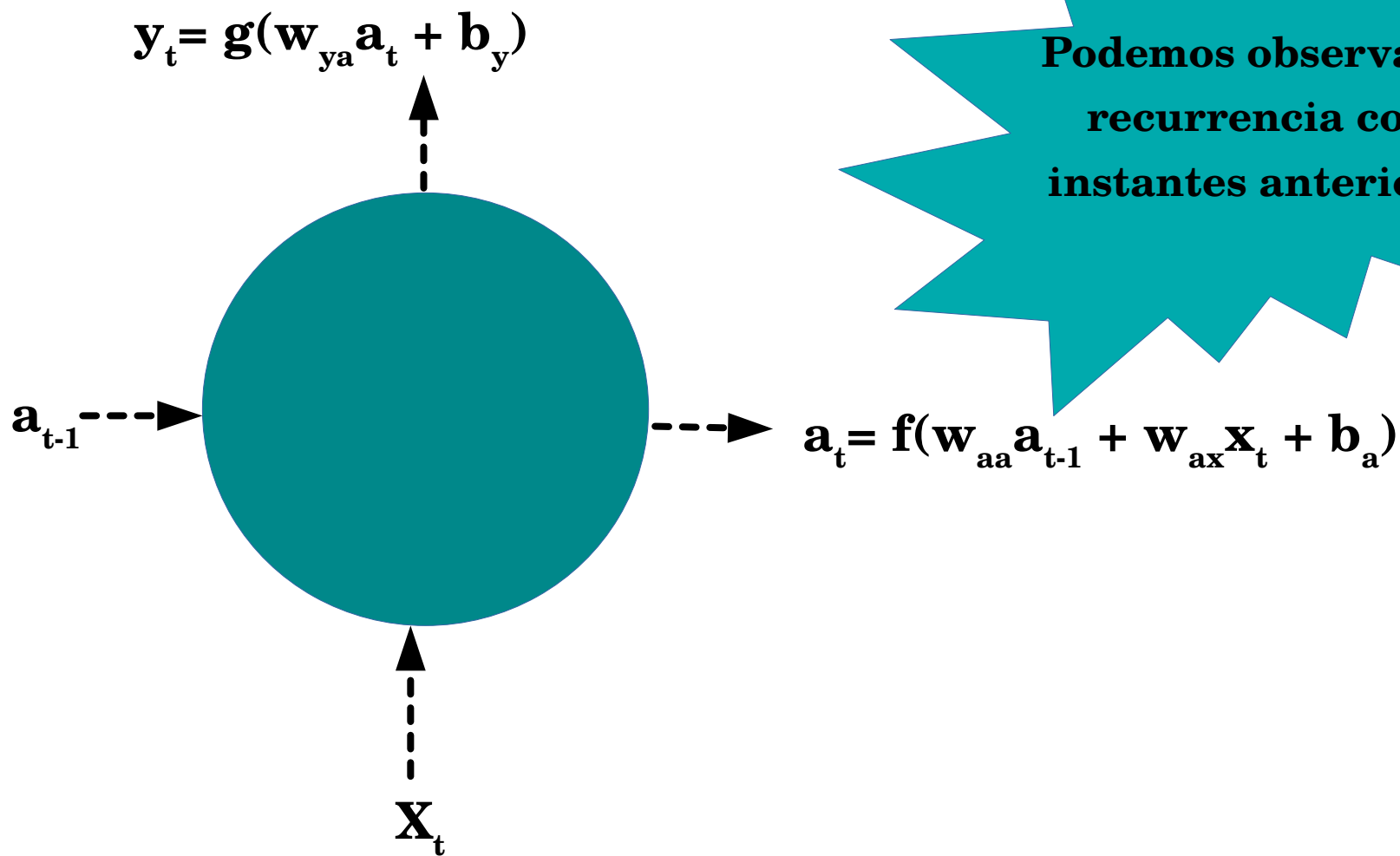
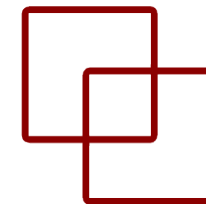
3. Funcionamiento (II)

RNN



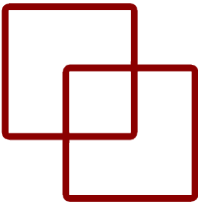
3. Funcionamiento (II)

RNN

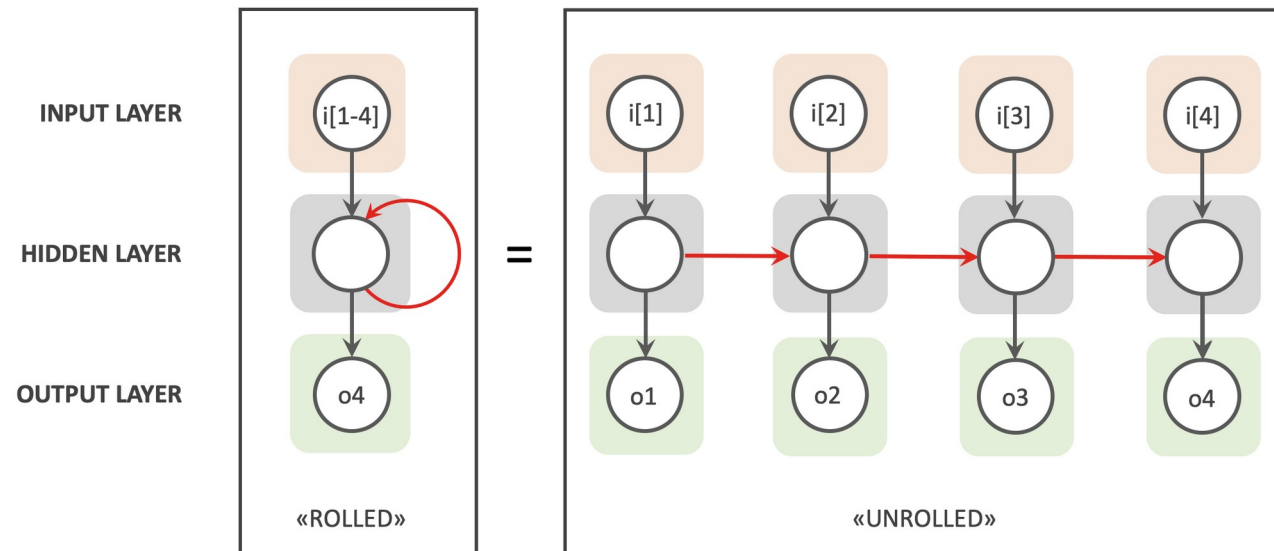


**Podemos observar la
recurrencia con
instantes anteriores**

3. Funcionamiento (II)

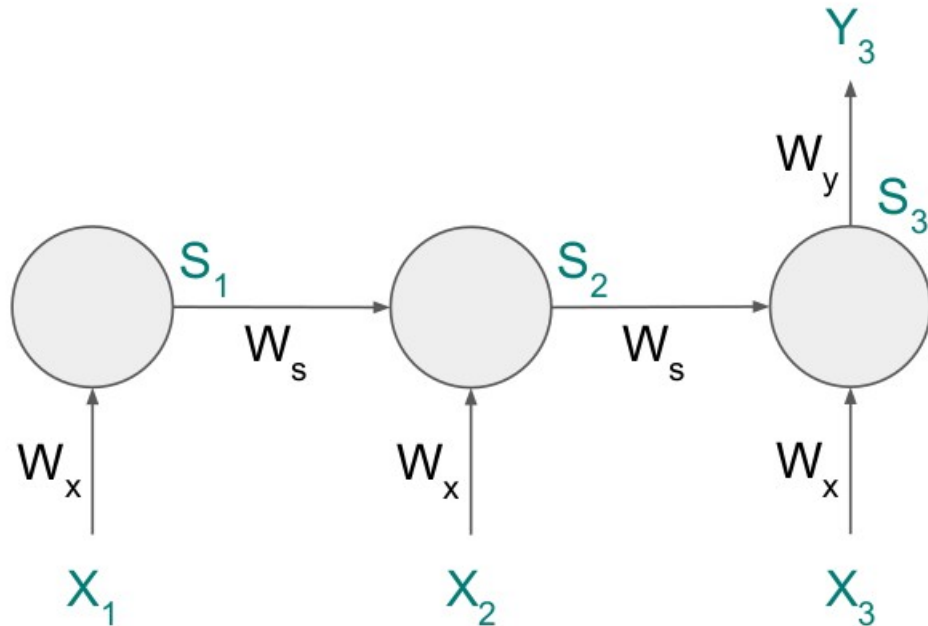
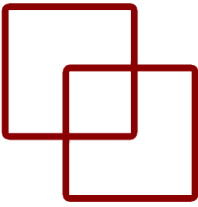


- **Backpropagation** con componentes temporales.
- El ajuste de los pesos se realiza con un proceso similar al de las redes *feed-forward*, pero considerando los pasos previos a través de la memoria.
 - Se calcula el error y se actualizan las matrices de **pesos** mediante las derivadas parciales del mismo.
 - La actualización dependerá de los **estados anteriores**.



3. Funcionamiento (III)

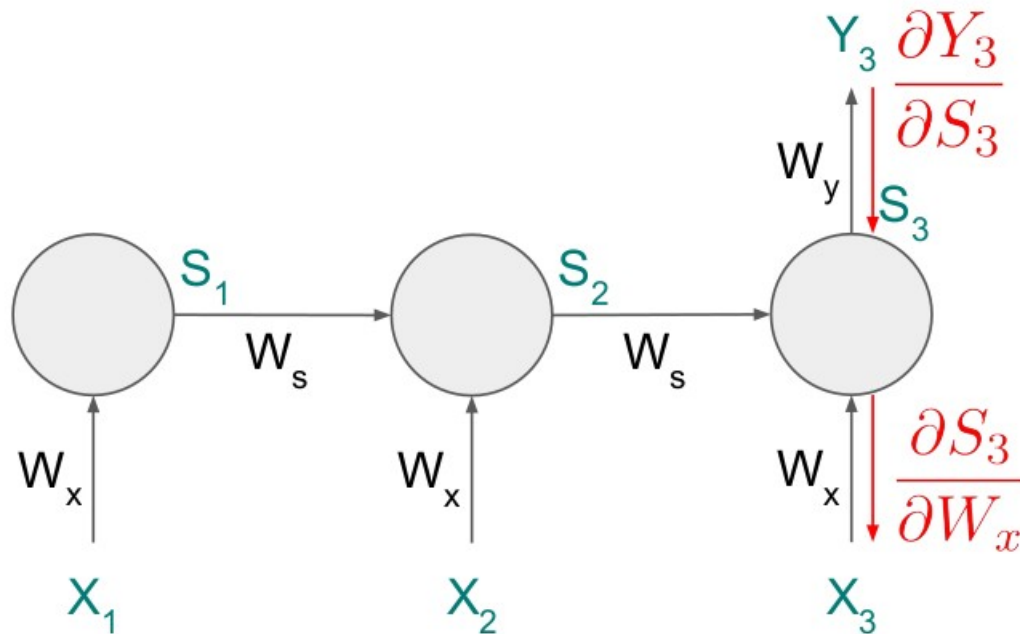
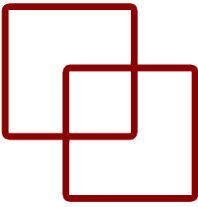
Ejemplo: Ajustar la matriz W_x en el instante $t=3$



$$\begin{aligned} \frac{\partial E_3}{\partial W_x} = & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_x} \end{aligned}$$

3. Funcionamiento (III)

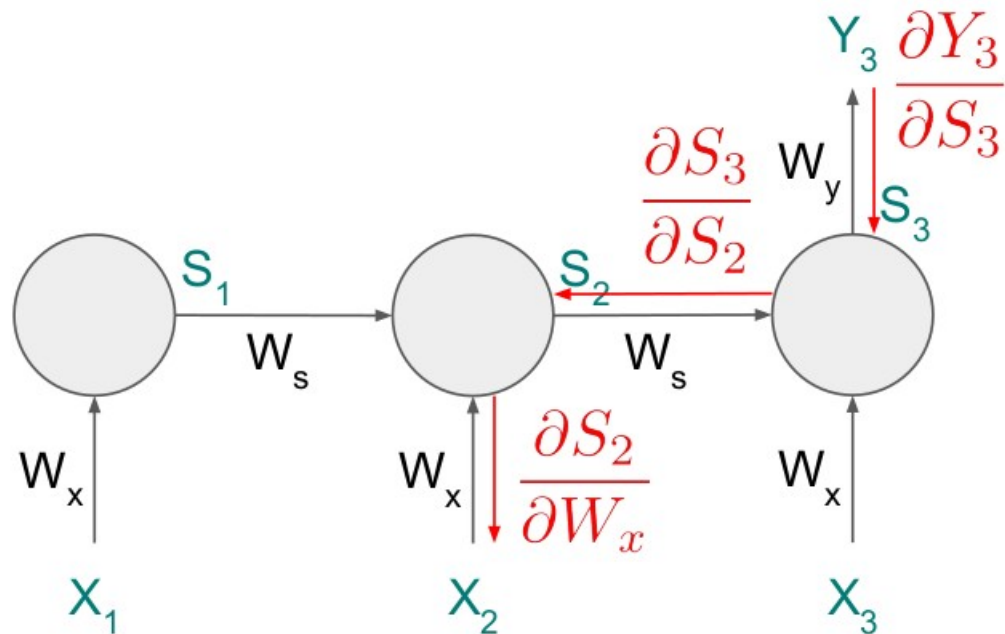
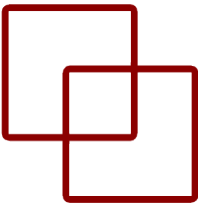
Ejemplo: Ajustar la matriz W_x en el instante $t=3$



$$\begin{aligned} \frac{\partial E_3}{\partial W_x} = & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_x} \end{aligned}$$

3. Funcionamiento (III)

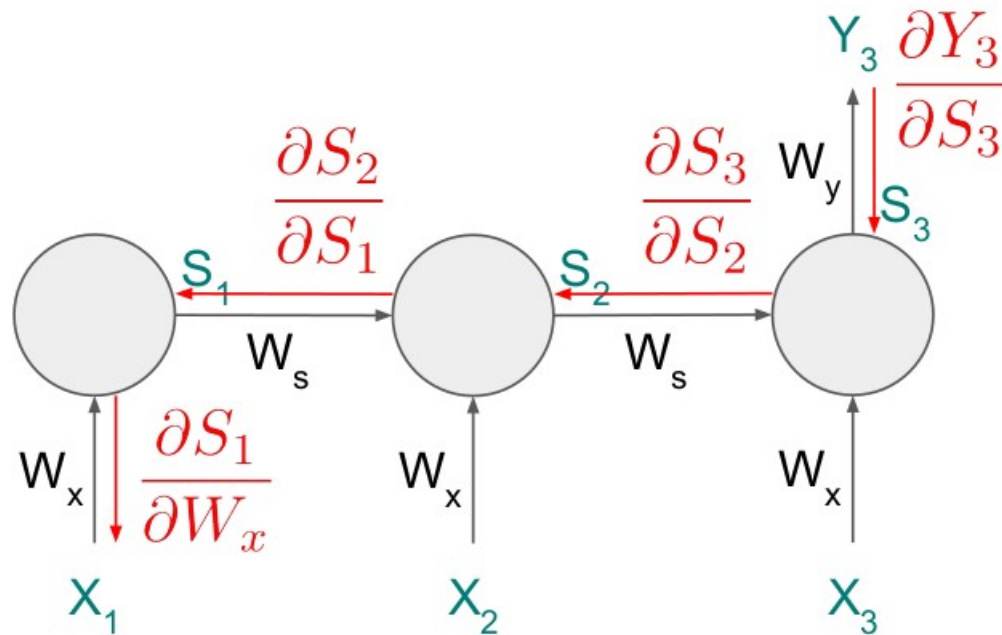
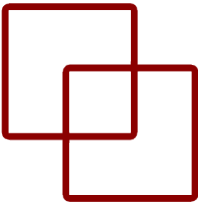
Ejemplo: Ajustar la matriz W_x en el instante $t=3$



$$\begin{aligned} \frac{\partial E_3}{\partial W_x} = & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_x} \end{aligned}$$

3. Funcionamiento (III)

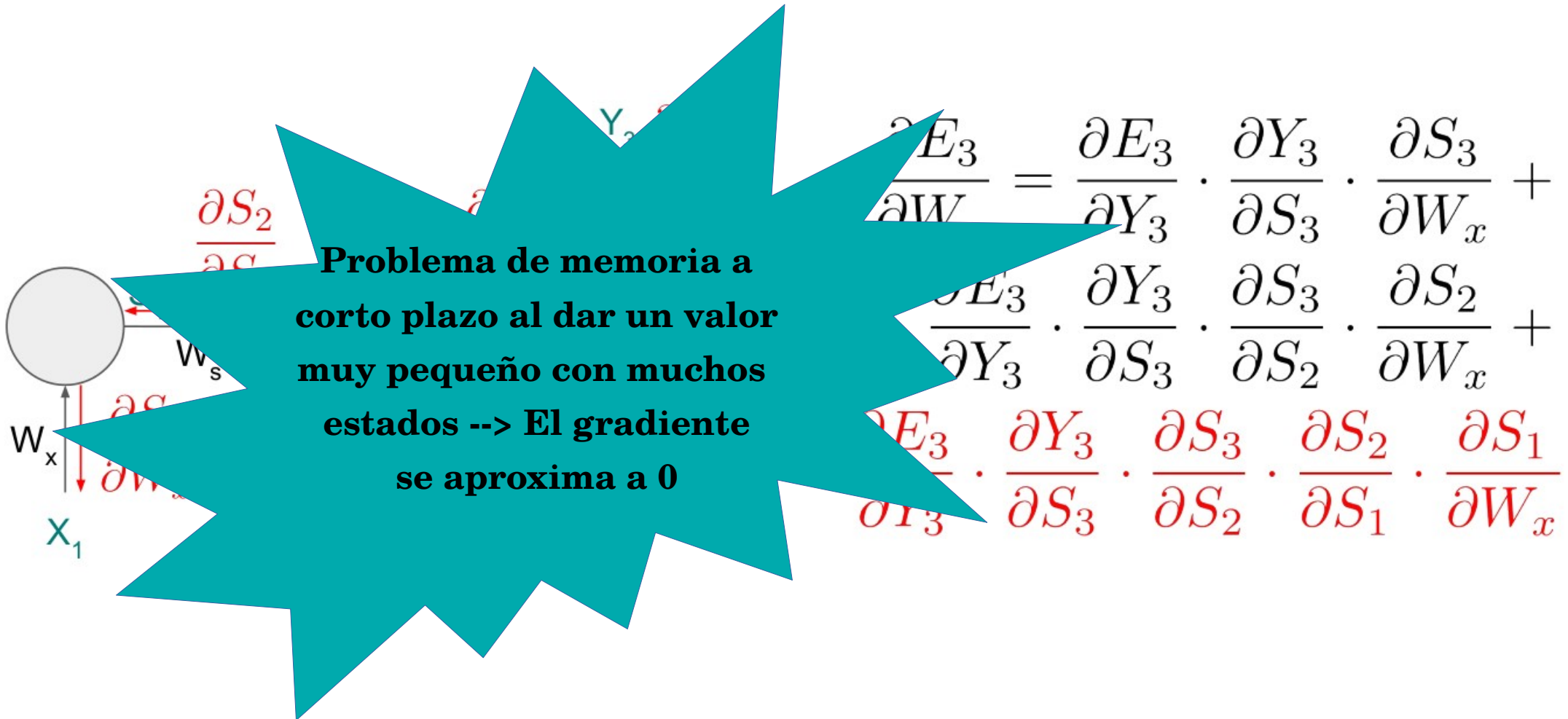
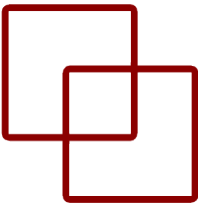
Ejemplo: Ajustar la matriz W_x en el instante $t=3$



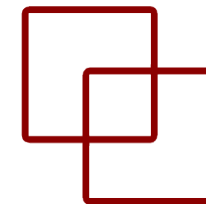
$$\begin{aligned} \frac{\partial E_3}{\partial W_x} = & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_x} + \\ & \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_x} \end{aligned}$$

3. Funcionamiento (III)

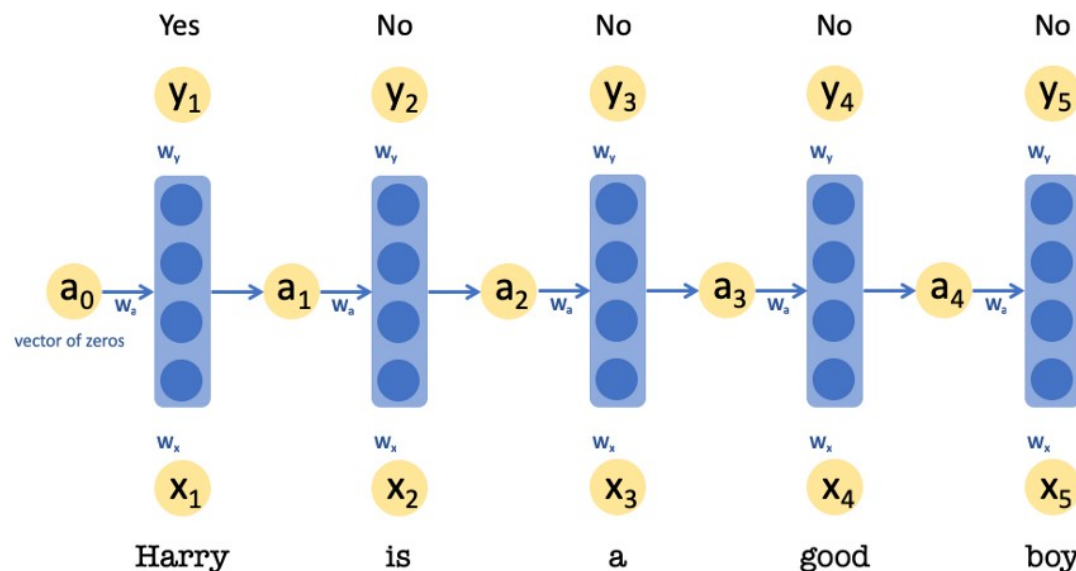
Ejemplo: Ajustar la matriz W_x en el instante $t=3$



3. Funcionamiento (IV)



- Problema de **memoria de corto plazo**
- El cálculo del gradiente para un instante dado considera los instantes anteriores. ¿Qué pasa si tenemos muchos estados anteriores?
 - Hasta 8 estados (más o menos), se puede calcular el gradiente. Con más estados el gradiente es demasiado pequeño
- **Problema de desvanecimiento del gradiente** (*vanishing gradient*).



Fuente: <https://mc.ai/explained-recurrent-neural-networks/>

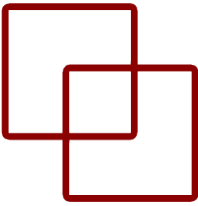


LAB CTIC  UNI

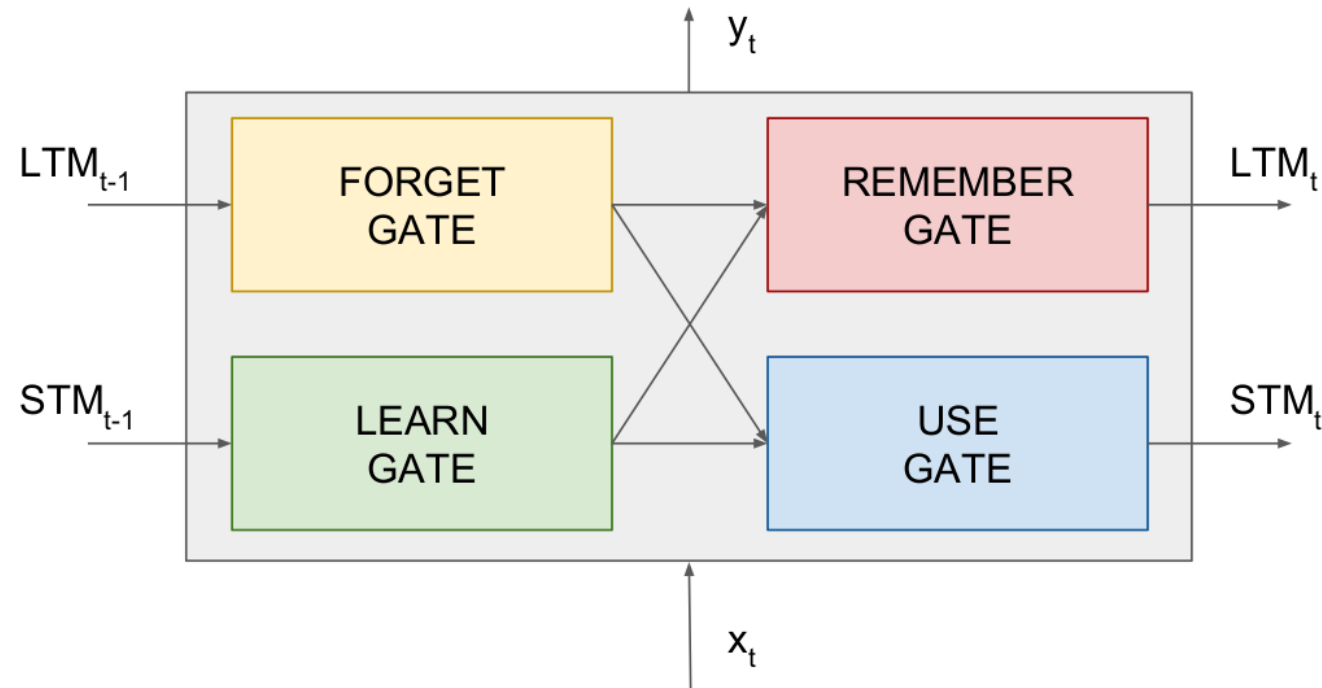
Redes Neuronales Recurrentes

LSTM

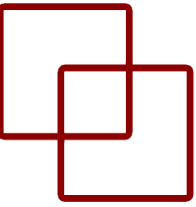
4. LSTM (I)



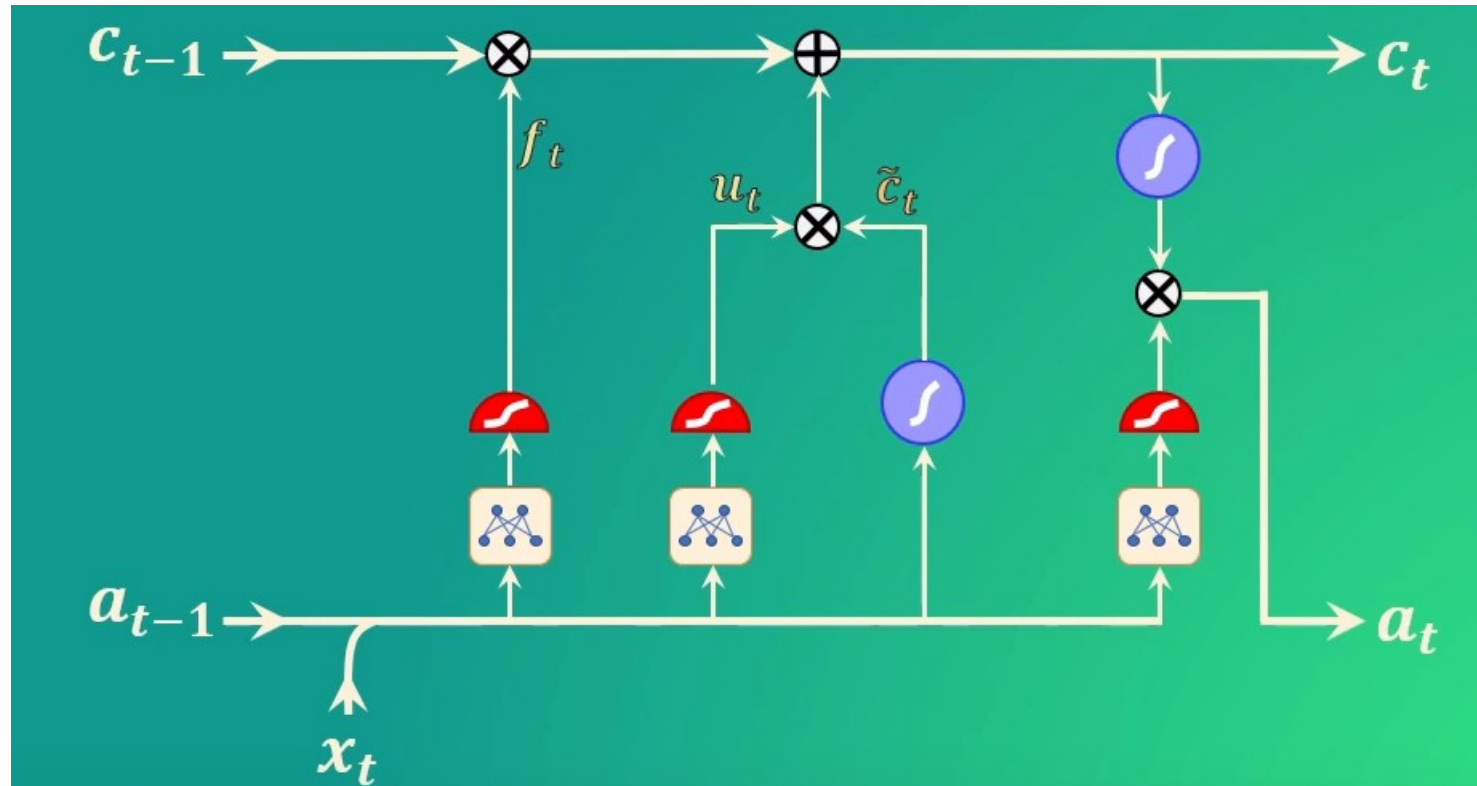
- Unidades más complejas para gestionar memoria a **corto plazo** y memoria a **largo plazo**.
 - Permite que el backpropagation pueda realizarse con una temporalidad mayor al descartar datos de corto/largo plazo
 - Consta de **4 puertas** para controlar el flujo de información:



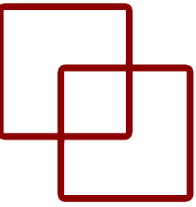
4. LSTM (II)



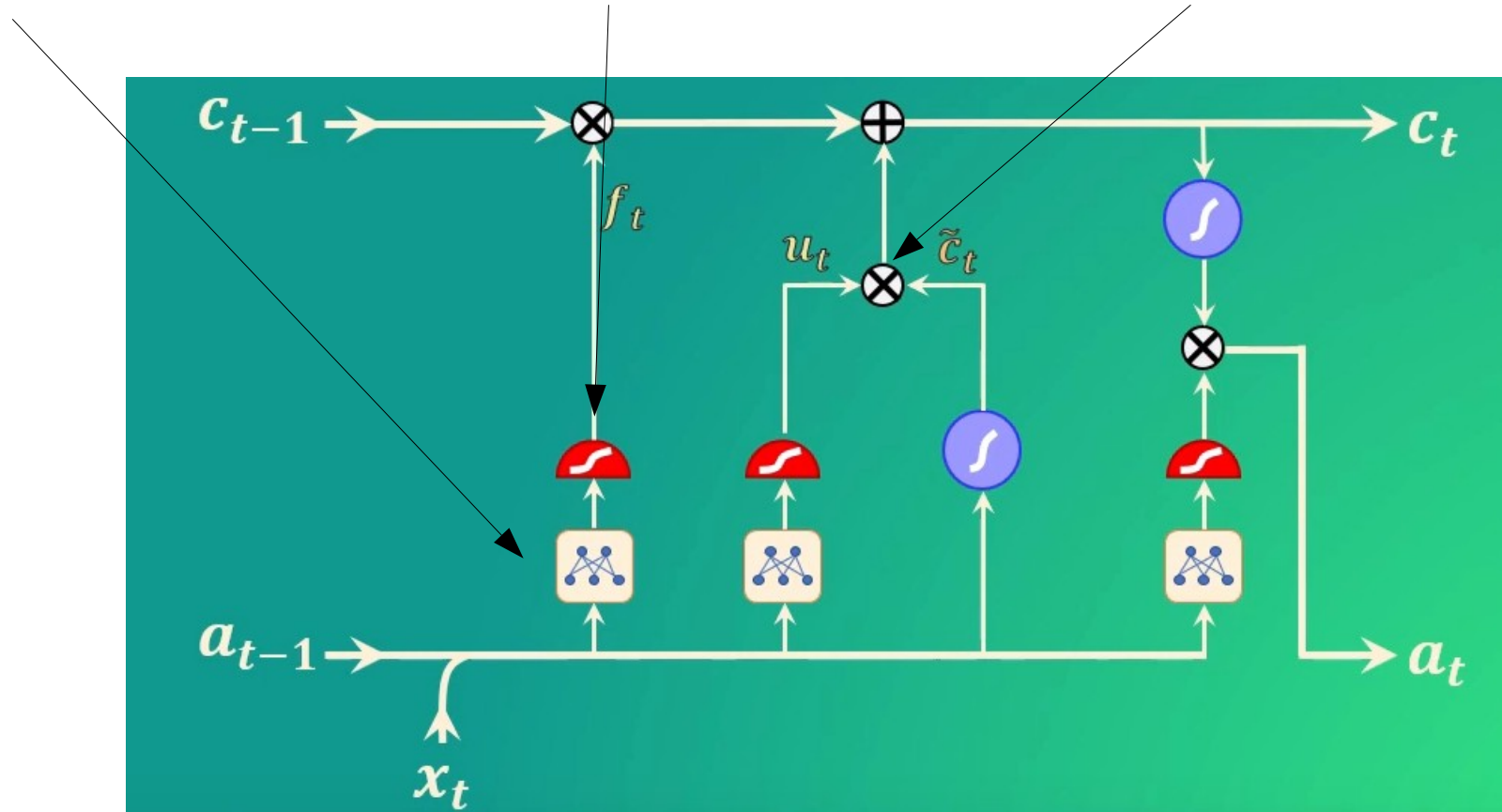
- Cada una de estas puertas están compuestas por una red neuronal, una función sigmoideal y un elemento multiplicador



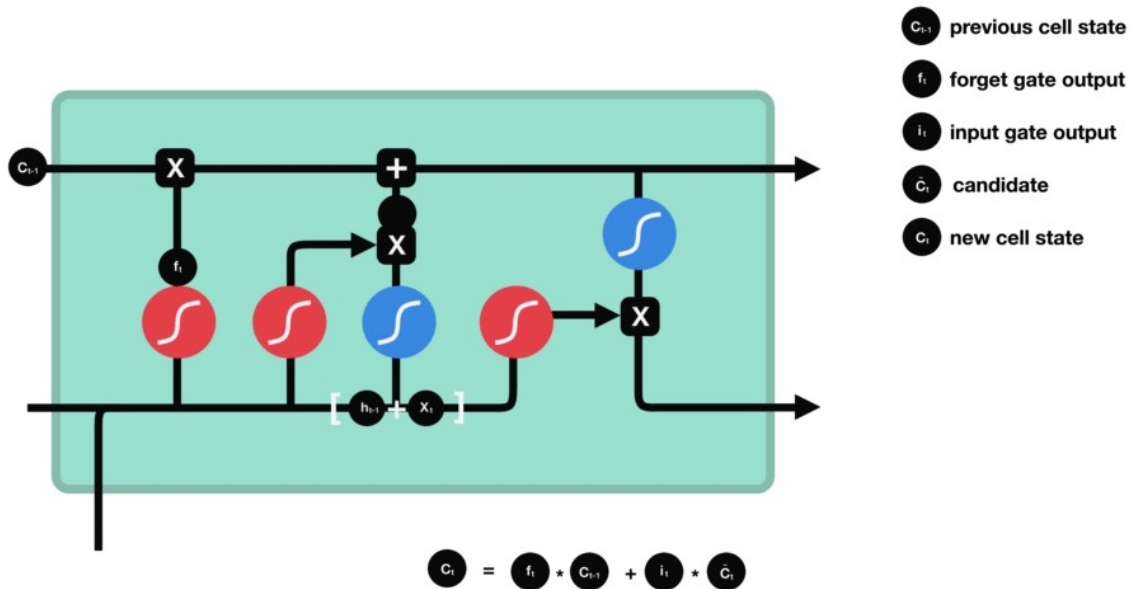
4. LSTM (II)



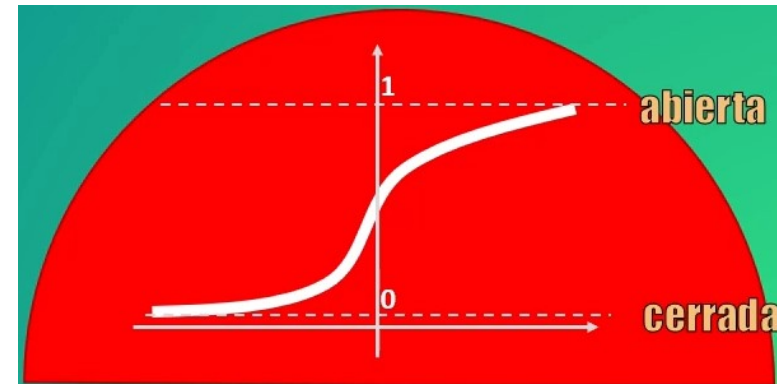
- Cada una de estas puertas están compuestas por una red neuronal, una función sigmoidal y un elemento multiplicador



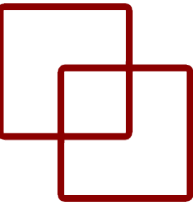
4. LSTM (III)



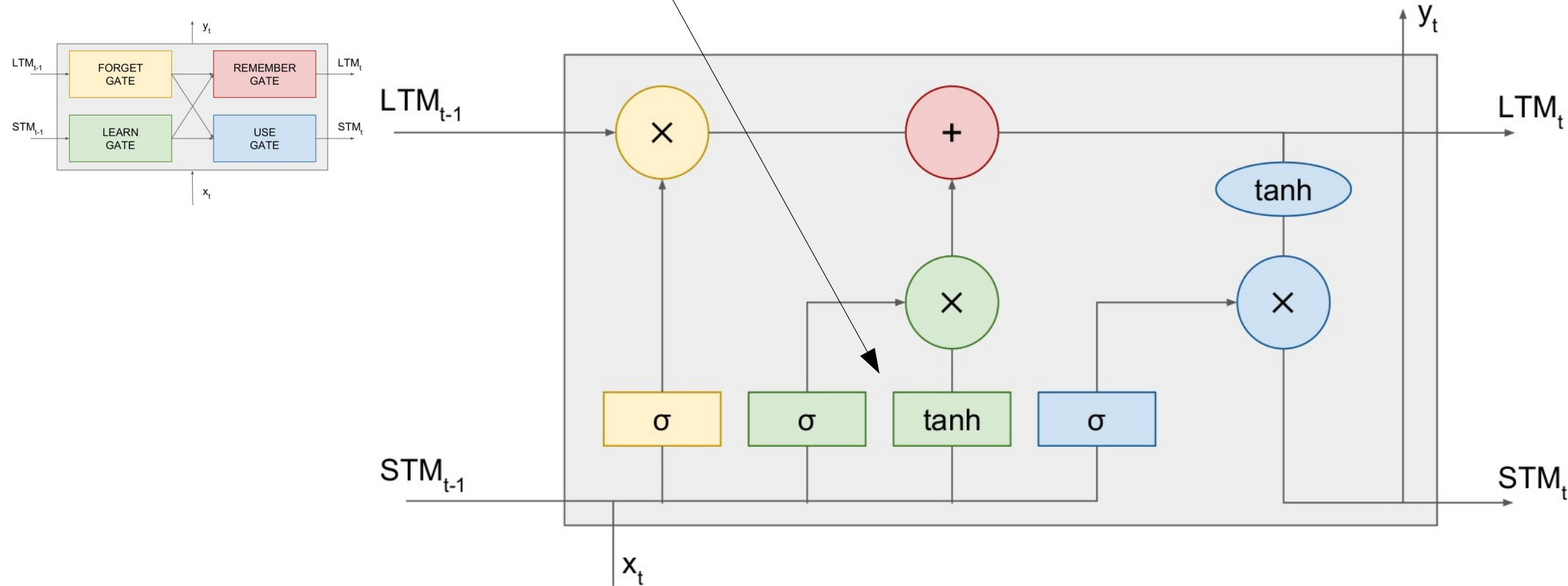
La función de activación le dará el comportamiento de válvula (si deja pasar el dato o no)



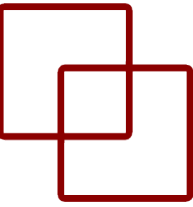
4. LSTM (IV)



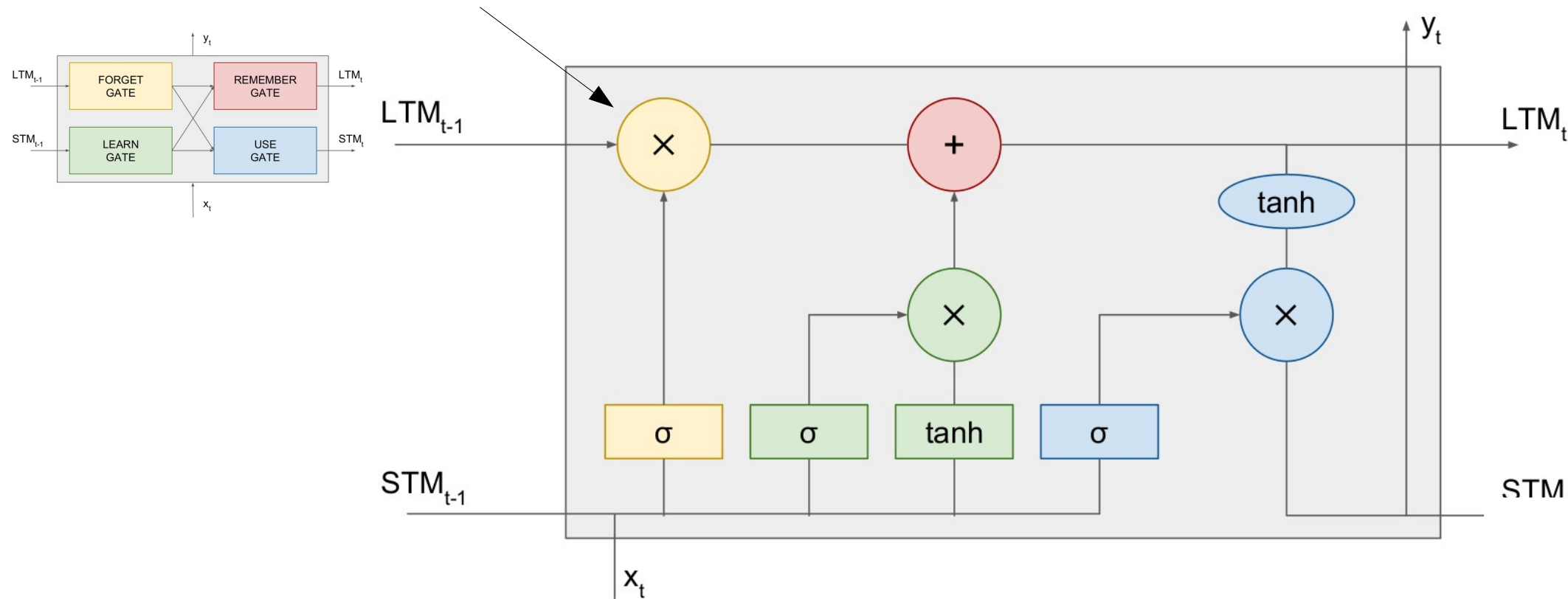
- **Learn gate:** Combina la entrada actual y la memoria a corto plazo.
 - La Sigmoide selecciona datos y establece cuales serán los que aprender



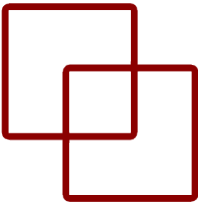
4. LSTM (V)



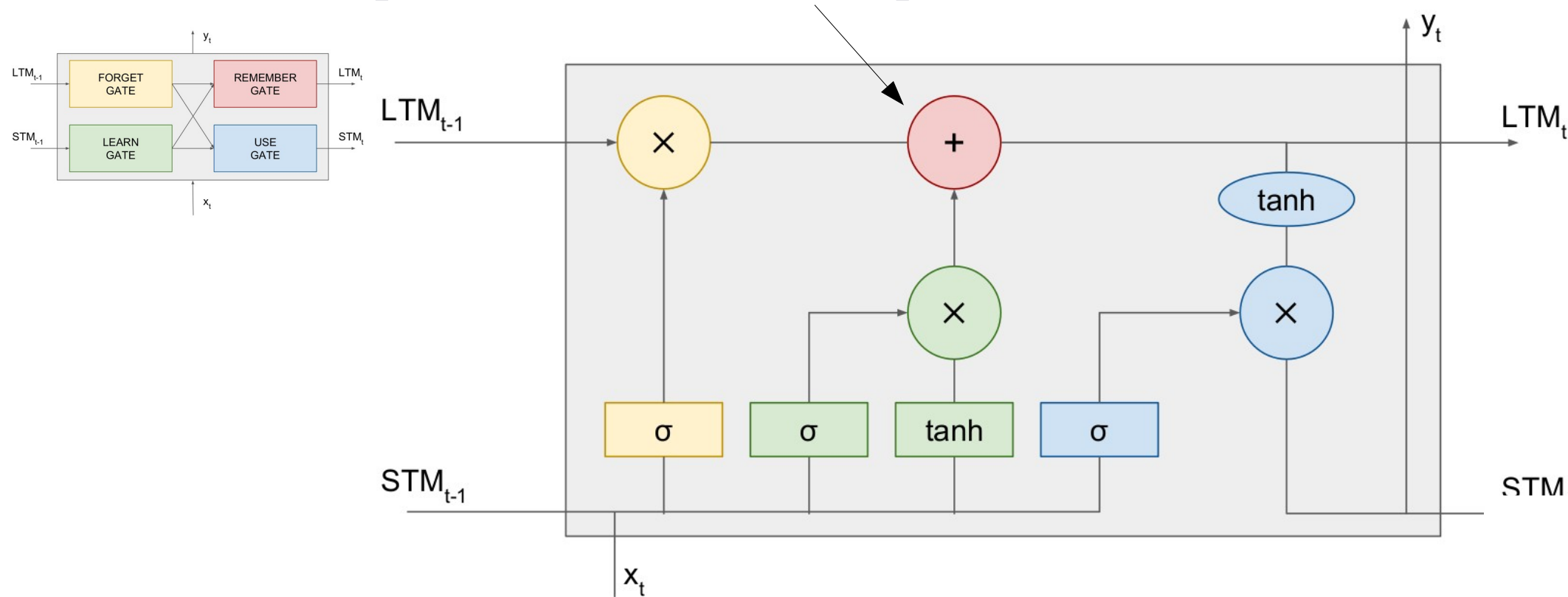
- **Forget gate:** Olvida parte de la entrada de la memoria a largo plazo (el “factor de olvido” depende de la entrada actual y la memoria a corto plazo).
 - Permite decir qué información descartar y no pasar a la celda de estado



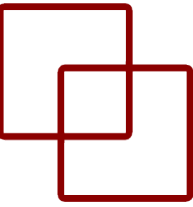
4. LSTM (VI)



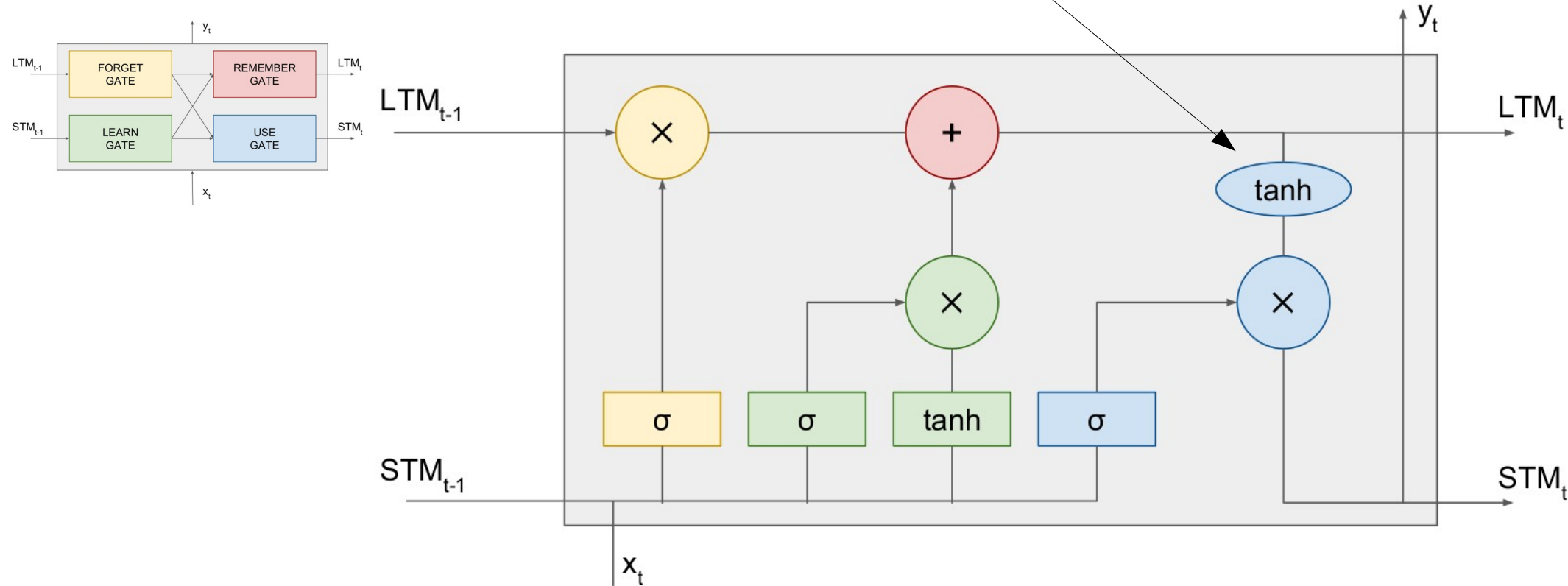
- **Remember gate:** Combina la memoria a corto plazo (salida de la *learn gate*) y la memoria a largo plazo (salida de la *forget gate*).
 - Decide qué recordar de las dos puertas anteriores



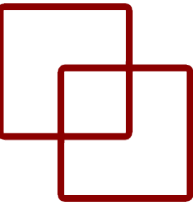
4. LSTM (VII)



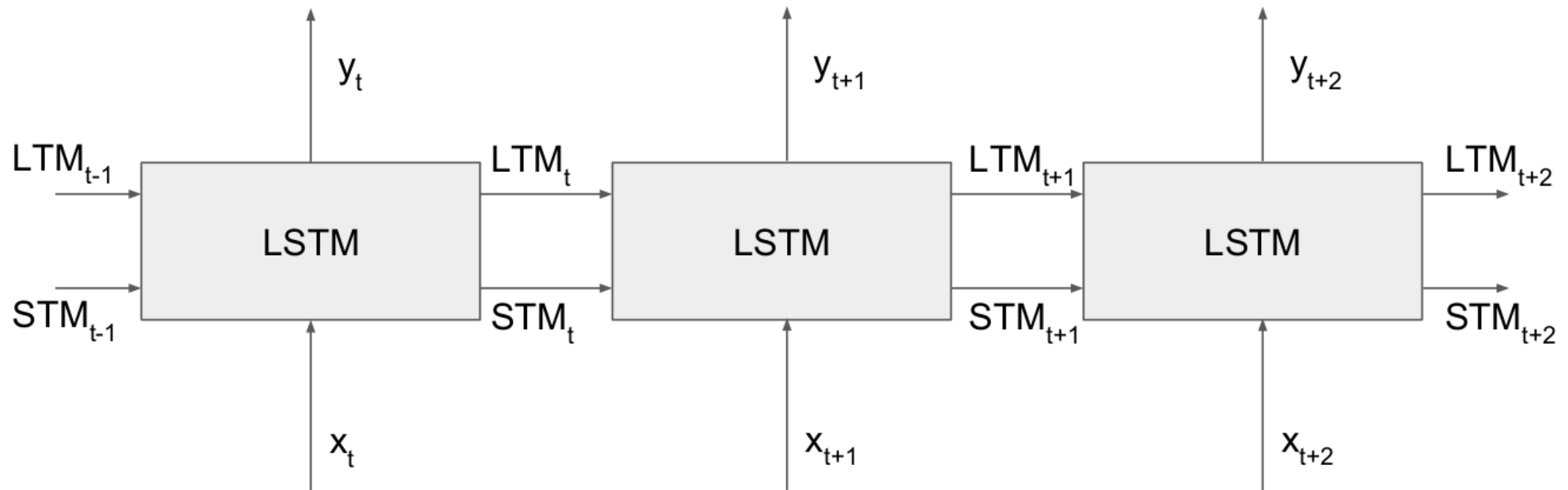
- **Use gate:** Combina la entrada actual, la memoria a corto plazo y la memoria a largo plazo (salida de *learn gate*) para generar la nueva memoria a corto plazo y la salida.



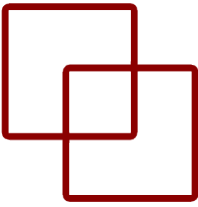
4. LSTM (VI)



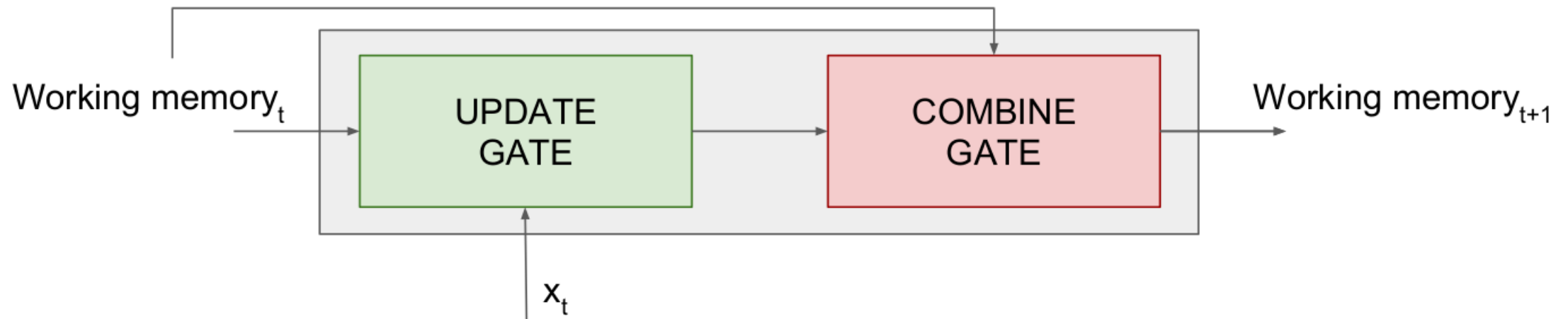
- La salida de memoria a corto plazo y memoria a largo plazo sirve para realimentar la siguiente entrada.



5. Otras arquitecturas



- **Gated Recurrent Unit (GRU)**
 - Utiliza una única memoria de trabajo.
 - Simplifica el procesamiento a dos “puertas”.

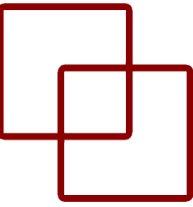




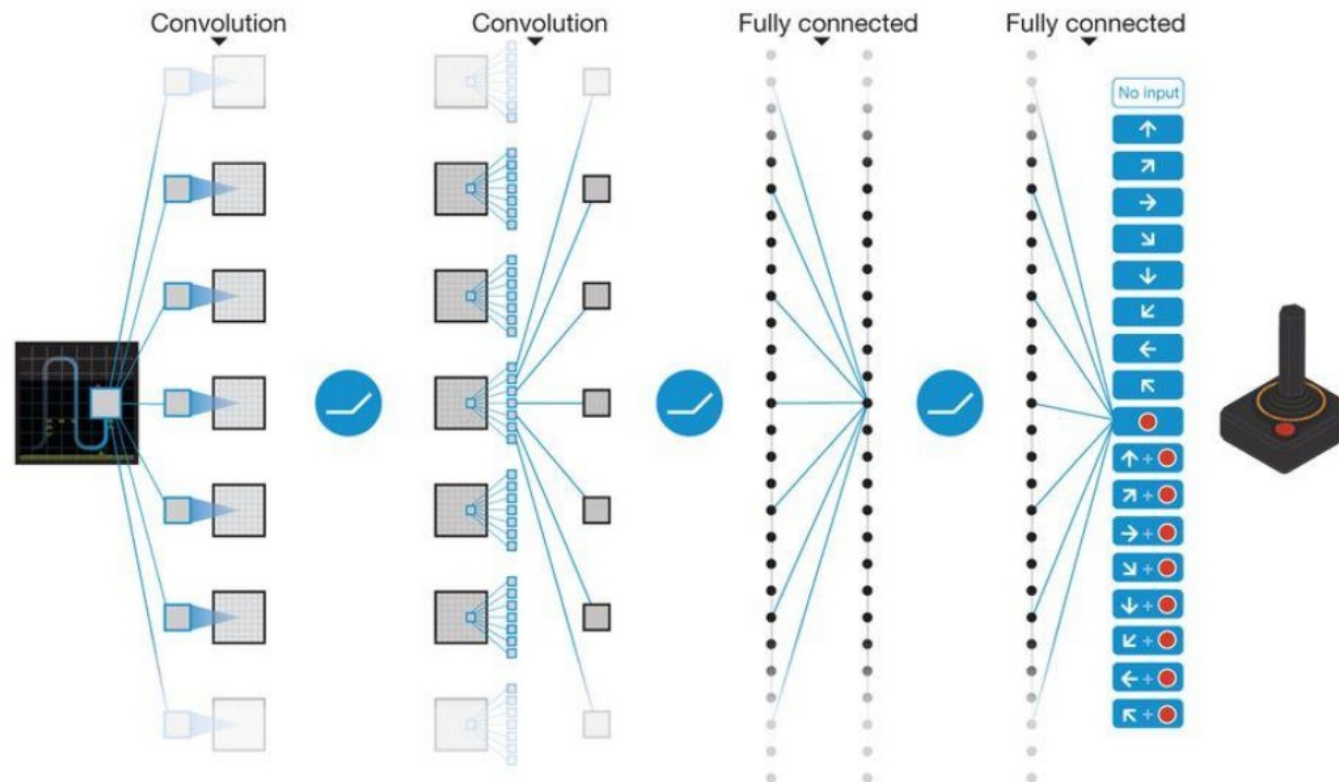
LAB CTIC  UNI

Deep Reinforcement Learning

1. Contexto (I)

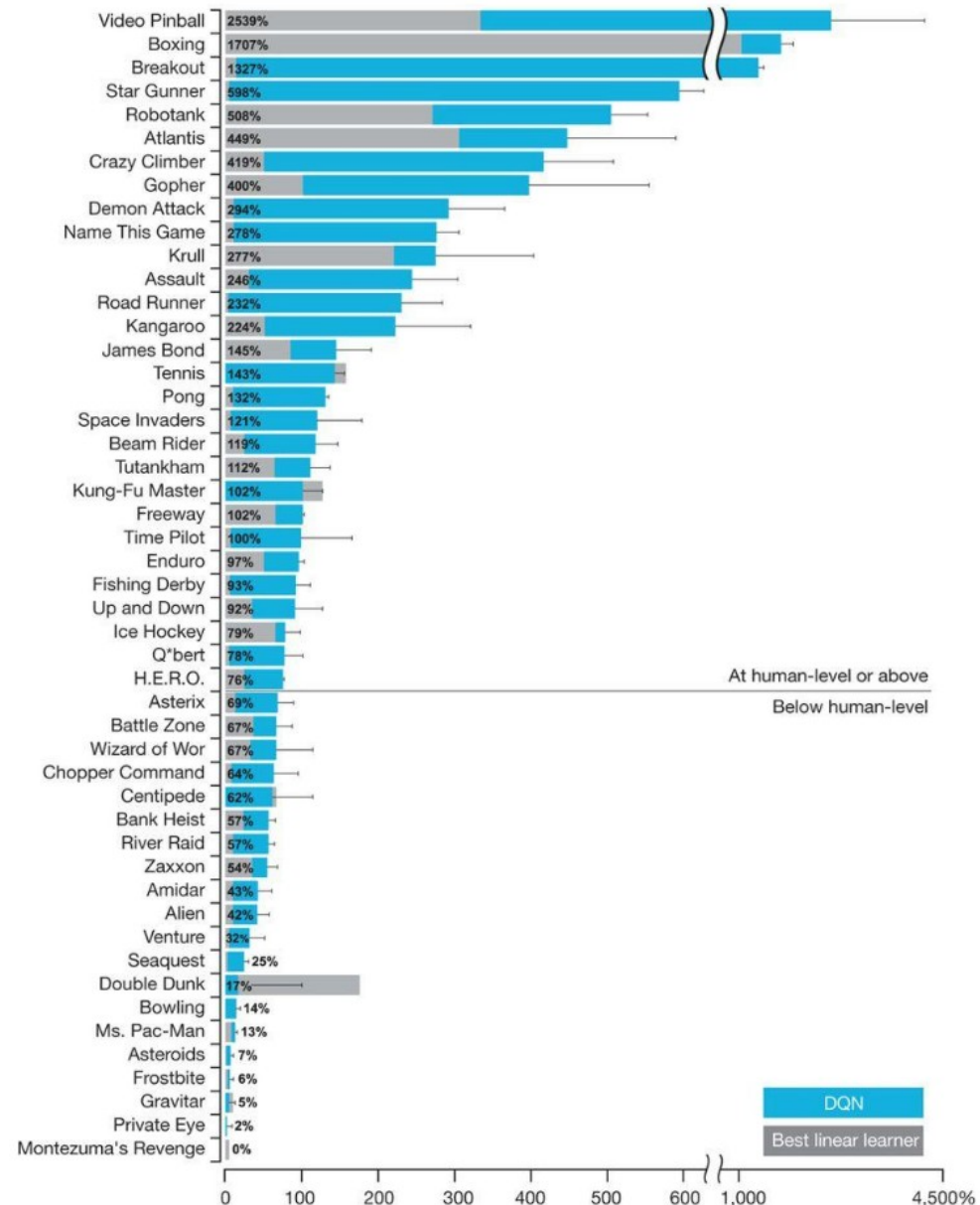


- V. Mnih et al. Playing Atari with Deep Reinforcement Learning. NIPS 2013 Deep Learning Workshop.

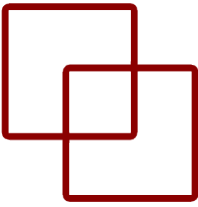


1. Contexto (II)

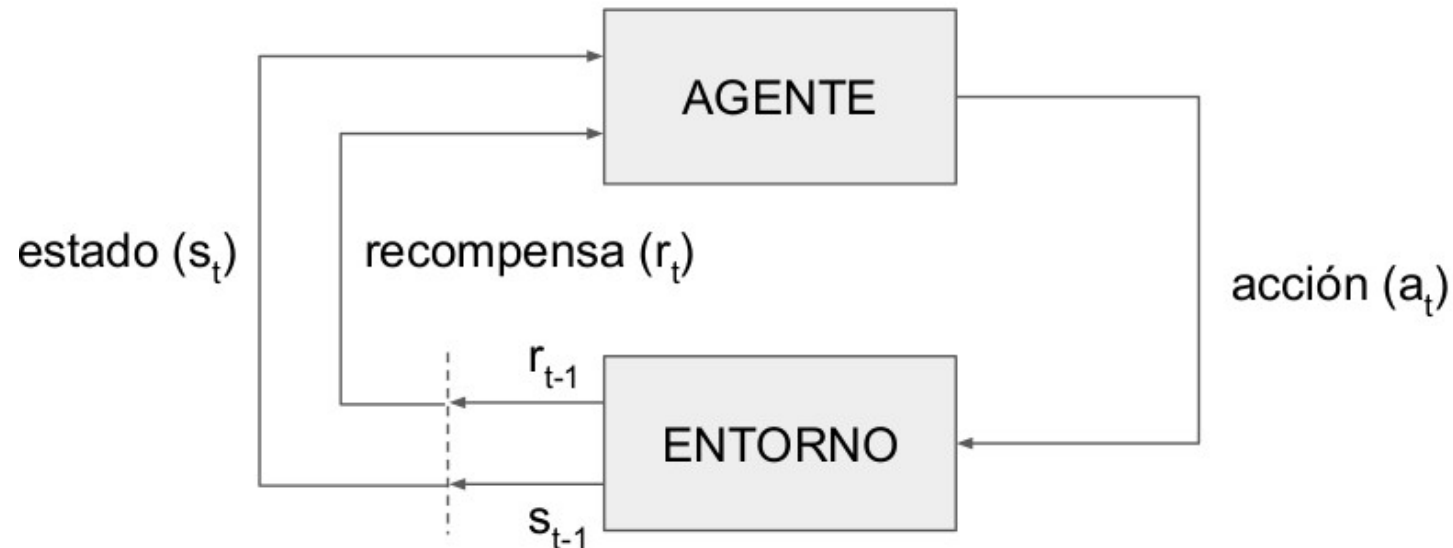
- Nivel similar al de un humano.
- Sin información previa.
- Sólo a partir de las imágenes.



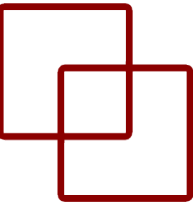
2. Introducción (I)



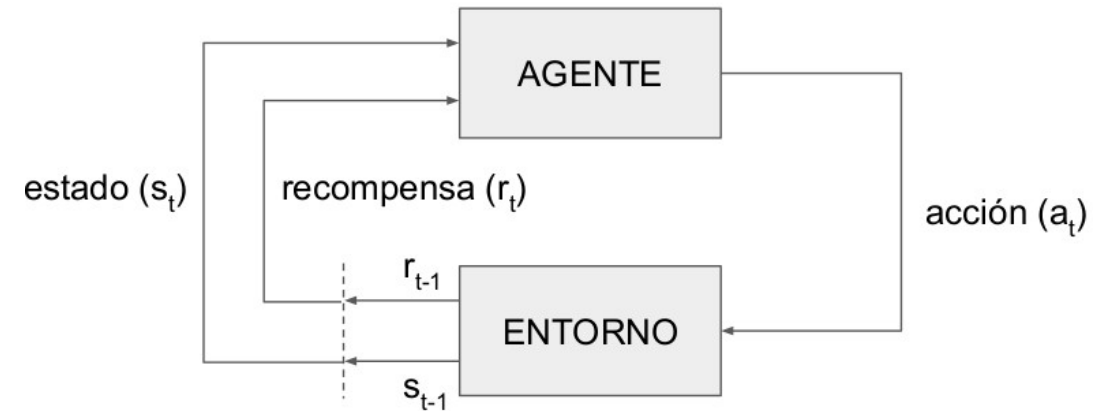
- El aprendizaje por refuerzo consiste en aprender a base de **prueba y error**.
- Al realizar una acción se obtiene una recompensa que indica lo correcta que fue dicha acción.
 - La recompensa será positiva si la acción fue correcta.
 - Negativa si fue incorrecta.



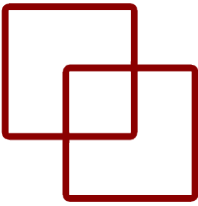
2. Introducción (II)



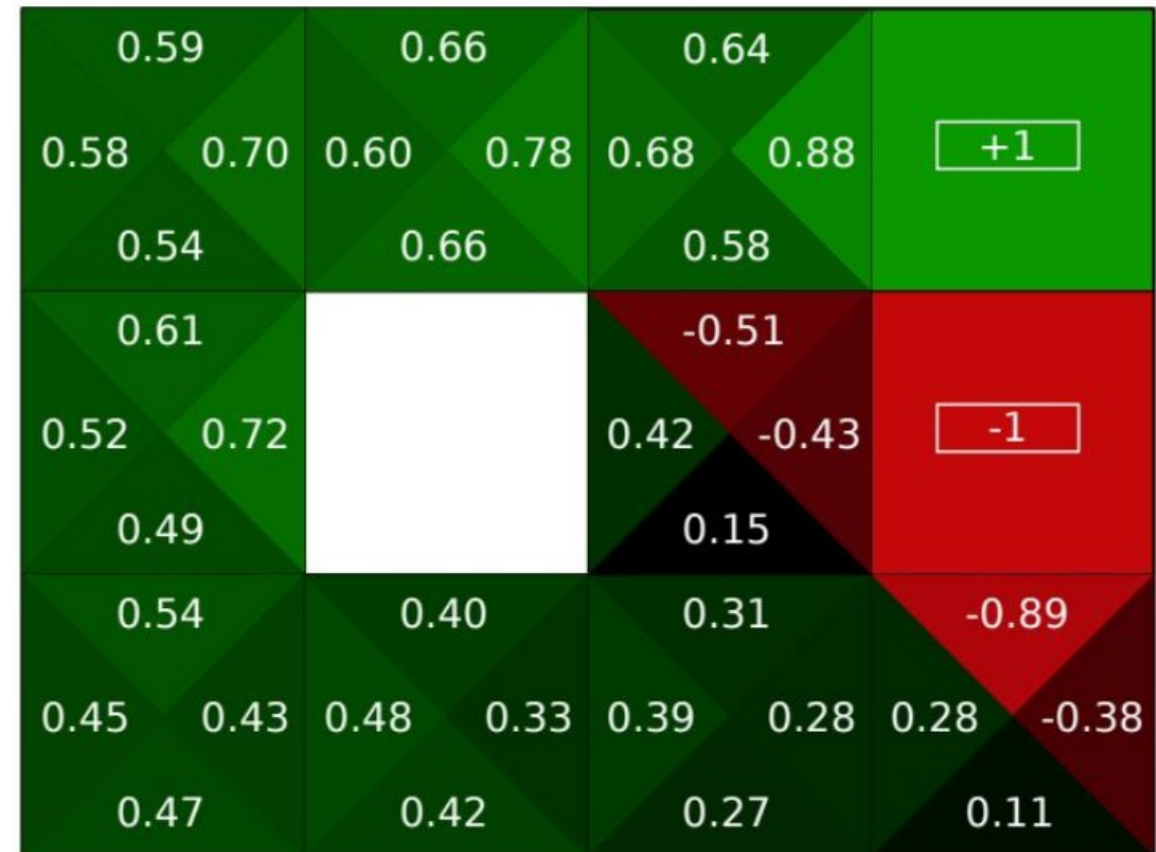
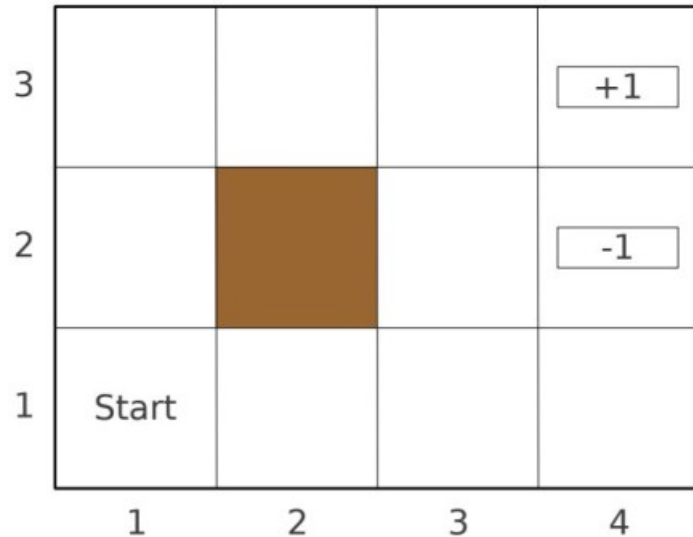
- Exploración vs. explotación.
 - Encontrar nuevas soluciones o aprovechar las ya conocidas.
- Tareas continuas vs. episódicas.
 - Tareas que continúan para siempre o con un inicio y final definidos.
- Política.
 - Acción a realizar en cada estado.
- El aprendizaje por refuerzo consiste en encontrar la **política óptima**.



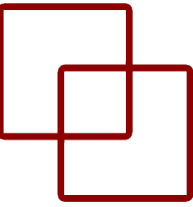
2. Introducción (III)



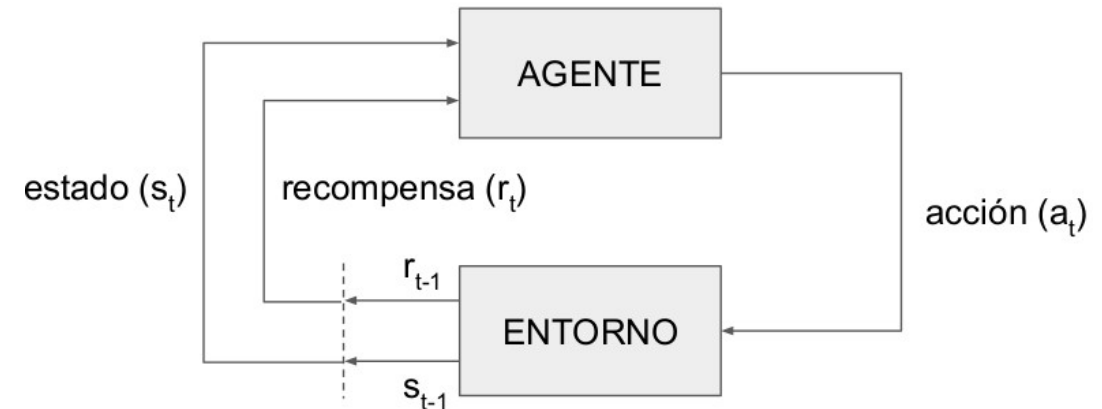
- La política óptima se calcula según el retorno esperado de cada par estado-acción
 - Función de valor.



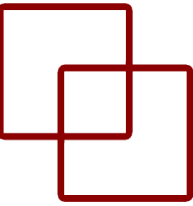
2. Introducción (IV)



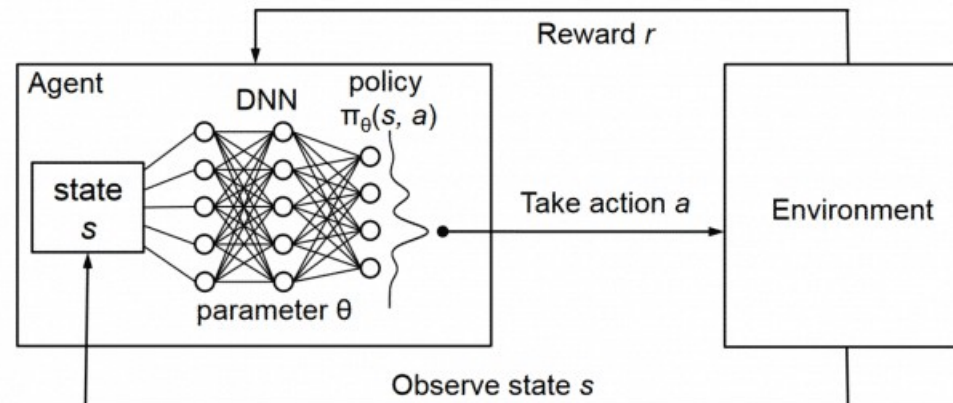
- Algoritmos de aprendizaje por refuerzo para **entornos discretos** (número finito de estados y acciones).
 - Iteración de valores.
 - Iteración de políticas.
 - Métodos Monte Carlo.
 - Sarsa.
 - Q-Learning.
- Soluciones para aprendizaje por refuerzo para **entornos continuos**.
 - Discretizar.
 - Aproximación de la función de valor (estado-acción).



3. Q-Networks (I)



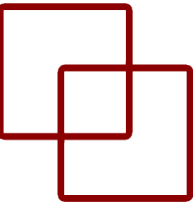
- Deep Reinforcement Learning se basa en el uso de una red neuronal para **aproximar** la función de valor.
 - La entrada de la red será la información del estado actual.
 - La salida será el retorno esperado para cada una de las posibles acciones en ese estado.
 - La red puede tener cualquier arquitectura: Feed-forward, CNNs, etc.
- Al utilizar una red neuronal se requiere una gran cantidad de datos para poder aproximar una política y, aún así, **no se puede garantizar** que se obtenga la política óptima.



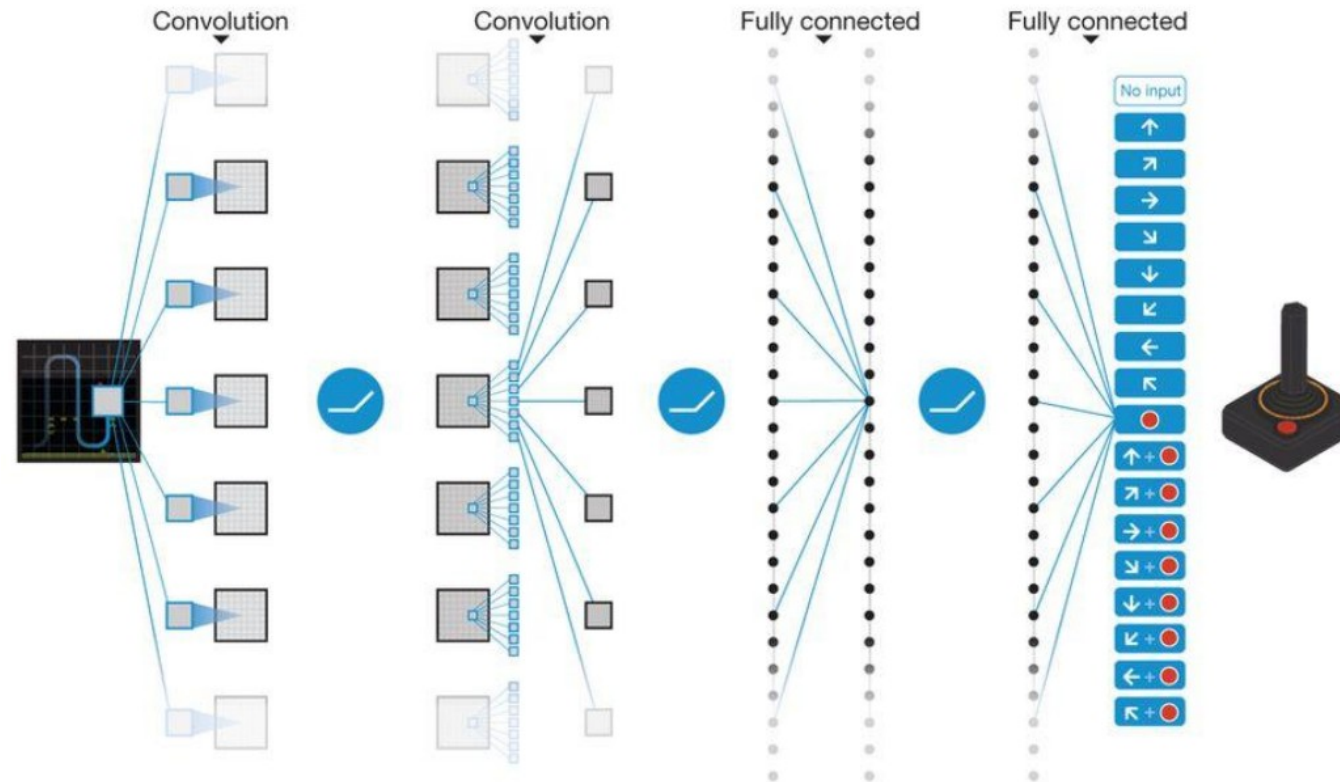
Fuente: <https://www.novatec-gmbh.de/en/blog/deep-q-networks/>

3. Q-Networks (II)

Ejemplo

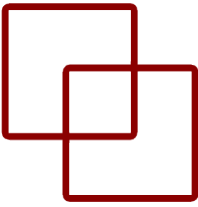


- CNN para aproximar el retorno de cada acción según el estado actual.

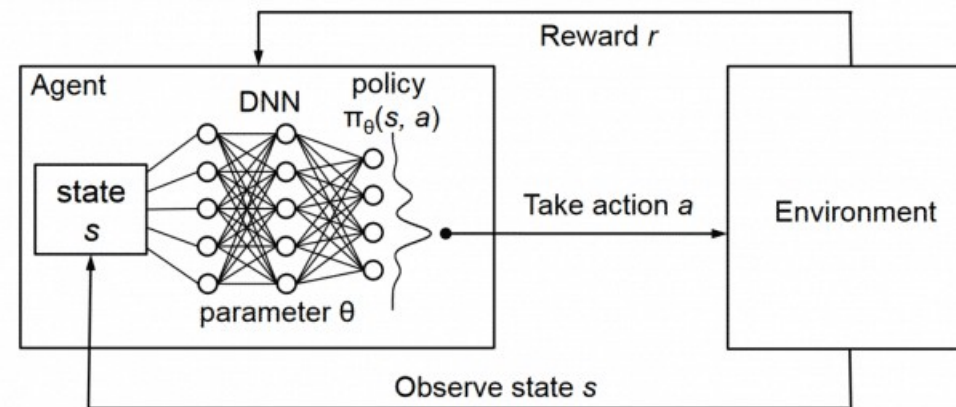


3. Q-Networks (III)

Optimización para entrenar DRL

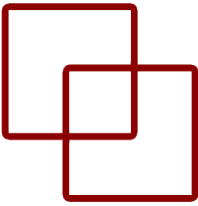


- Repetición de experiencias (*experience replay*).
 - Experiencia: Tupla $(s_t, a_t, r_{t+1}, s_{t+1})$
 - Se almacenan en un buffer de experiencias.
 - Cada batch de entrenamiento se forma seleccionando experiencias aleatorias del buffer.
 - Aleatorias → Disminuye la correlación entre muestras.
 - Permite repetir tuplas.
 - Se puede dar prioridad a experiencias más relevantes.

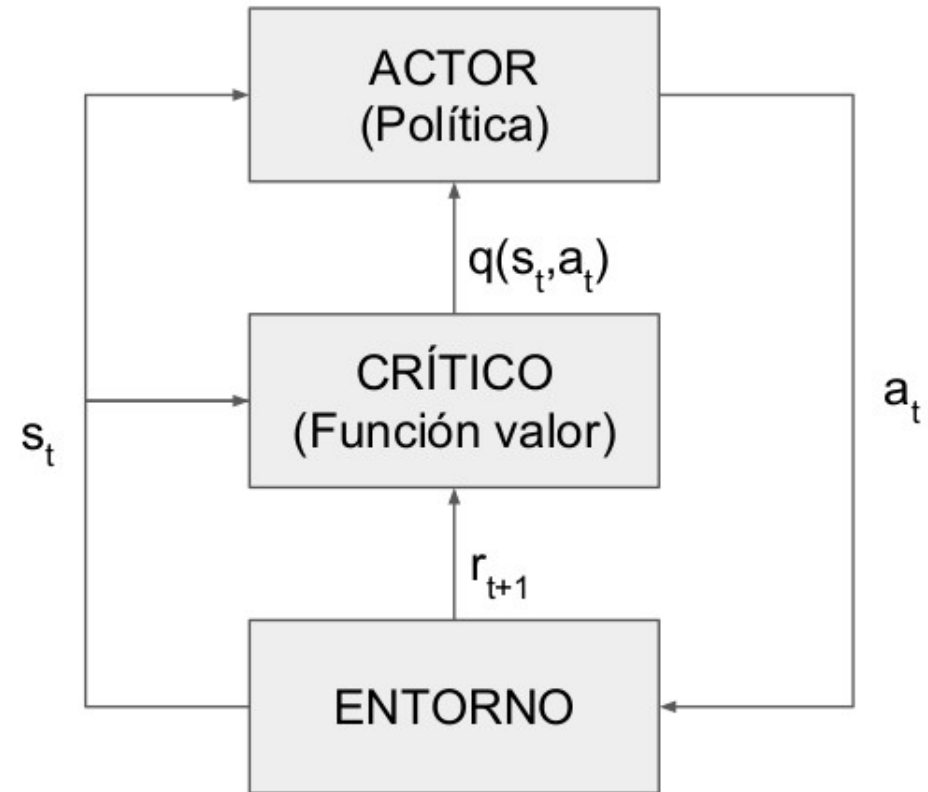


Fuente: <https://www.novatec-gmbh.de/en/blog/deep-q-networks/>

4. Algoritmos Actor-Crítico

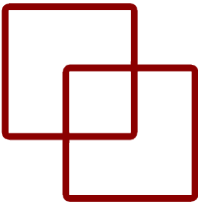


- El agente está formado por dos modelos que interaccionan.
 - **Actor:** Ejecuta y actualiza la política.
 - **Crítico:** Da el valor de la acción realizada.



5. Aplicaciones

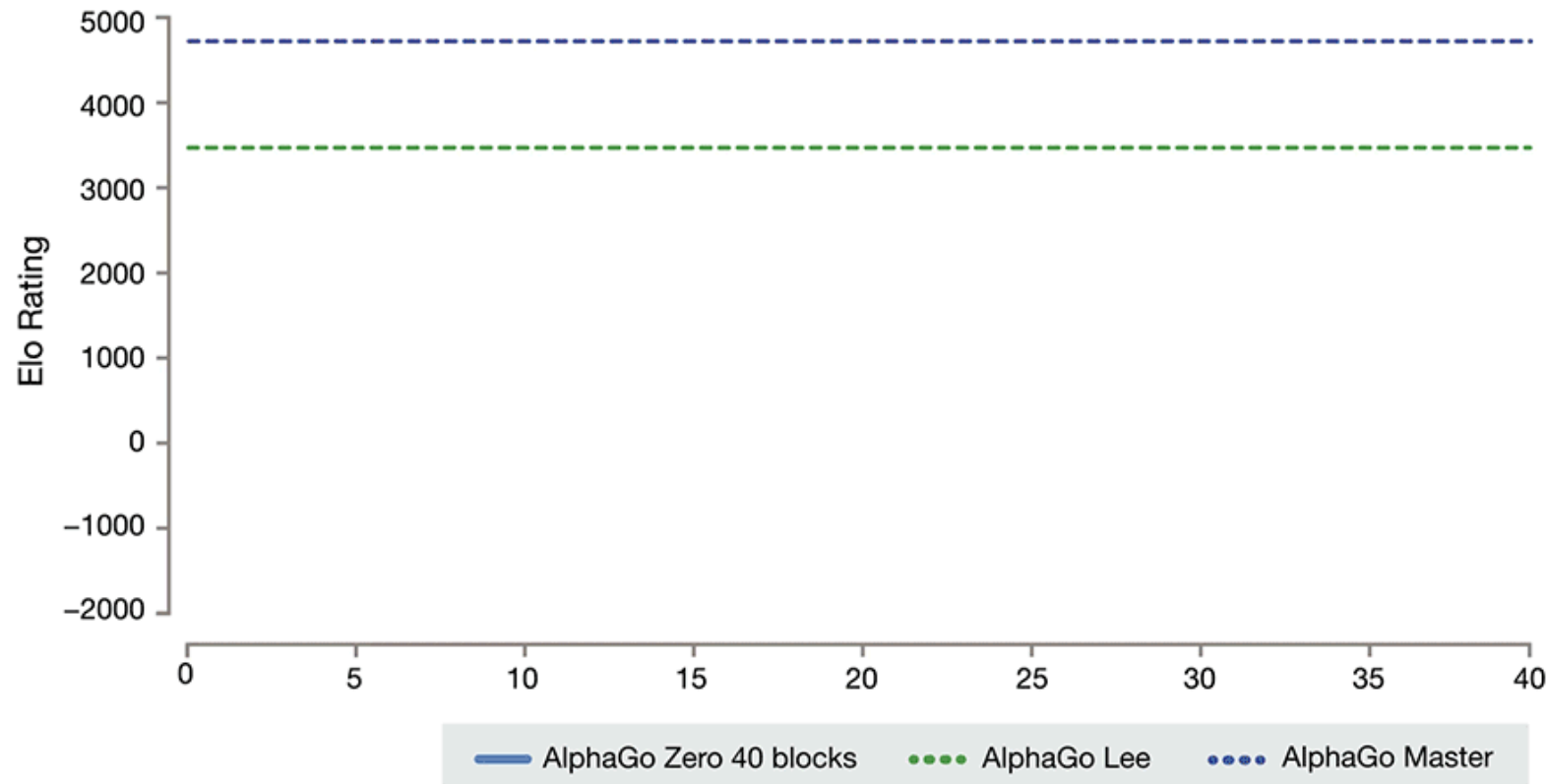
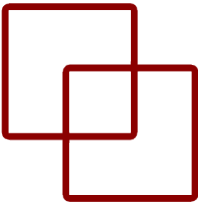
Juegos de Atari



Fuente: <https://deepmind.com/research/publications/playing-atari-deep-reinforcement-learning>

5. Aplicaciones

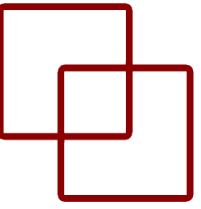
AlphaGo Zero



Fuente: <https://deepmind.com/blog/article/alphago-zero-starting-scratch>

5. Aplicaciones

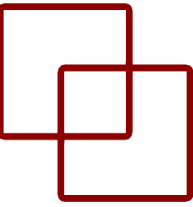
Aprender a moverse



Fuente: <https://deepmind.com/blog/article/producing-flexible-behaviours-simulated-environments>



Conclusiones



1. Conclusiones

- Hemos visto una pequeña introducción al mundo del Deep Learning.
 - Redes neuronales feed-forward
 - Redes convolucionales
- Un campo en auge y en constante evolución.
- Fundamental para cualquier persona que se dedique al machine learning.

¡GRACIAS!



Dr. Manuel Castillo-Cara
Deep Learning con Keras y Python
Web: www.smartcityperu.org